

Efficient Time-Restricted k NN Search in High-Dimensional Data Using Multi-Level Block Indexing, with Extensions to Multi-Attribute Filtering

JISOO KANG, CHANGHUN HAN, SUJI KIM, and HA-MYUNG PARK, Kookmin University, Seongbuk-gu, South Korea

How can we efficiently index extensive high-dimensional vector data increasing over time, enabling quick and accurate proximity searches within designated time windows? A time-restricted k -Nearest Neighbor (TkNN) query aims to identify the k -nearest vectors to a query vector within a specified time window. While high-dimensional and time-accumulating data are ubiquitous and managing such data efficiently is becoming increasingly significant, TkNN search within this context has not received much attention so far. In this article, we propose Multi-Level Block Indexing (MBI), a tailored indexing method for efficient approximate TkNN search. MBI employs an incremental hierarchical index structure that divides the data into multiple blocks based on timestamps. This structure ensures efficient query processing, irrespective of the length of the query time window, and facilitates the addition of new data over time. Furthermore, we extend TkNN beyond timestamps to the m -AkNN problem, incorporating m additional attributes—such as age, height, and weight in medical data or citation counts in academic papers—allowing queries to integrate multiple numerical constraints. Experimental results highlight MBI's superiority over conventional methods, achieving query processing speeds up to 10.88 times faster and offering logarithmic scaling in data insertion time as the data volume grows. Additionally, in m -AkNN queries, MBI maintains stable indexing performance and achieves up to 1.99 times faster query performance, demonstrating its effectiveness in large-scale, multi-attribute k NN search.

CCS Concepts: • **Information systems** → **Proximity search**; *Multimedia and multimodal retrieval*; • **Theory of computation** → *Data structures design and analysis*;

Additional Key Words and Phrases: k -Nearest Neighbor Search, Spatio-temporal Data, High-Dimensional, Time-Restricted k -Nearest Neighbor Search, Multi-Attribute Filtering

Associate Editor: Yanjie Fu

ACM Reference format:

Jisoo Kang, Changhun Han, Suji Kim, and Ha-Myung Park. 2026. Efficient Time-Restricted k NN Search in High-Dimensional Data Using Multi-Level Block Indexing, with Extensions to Multi-Attribute Filtering. *ACM Trans. Knowl. Discov. Data.* 20, 3, Article 41 (February 2026), 30 pages.

<https://doi.org/10.1145/3789265>

This work was supported by the Institute of Information and Communications Technology Planning and Evaluation (IITP) grant funded by the Korea government (MSIT) (No. RS-2025-02219317, AI Star Fellowship (Kookmin University)).

Authors' Contact Information: Jisoo Kang, Kookmin University, Seongbuk-gu, South Korea; e-mail: tn1078@kookmin.ac.kr; Changhun Han, Kookmin University, Seongbuk-gu, South Korea; e-mail: codingnoye@kookmin.ac.kr; Suji Kim, Kookmin University, Seongbuk-gu, South Korea; e-mail: suji2924@kookmin.ac.kr; Ha-Myung Park (corresponding author), Kookmin University, Seongbuk-gu, South Korea; e-mail: hmpark@kookmin.ac.kr.



This work is licensed under Creative Commons Attribution International 4.0.

© 2026 Copyright held by the owner/author(s).

ACM 1556-472X/2026/2-ART41

<https://doi.org/10.1145/3789265>

1 Introduction

How can we index extensive and time-accumulating vector data to enable fast and accurate proximity search in specific time windows? Time-accumulating data, which grow over time fastly, are ubiquitous. For example, GK2A, a weather satellite of South Korea, takes 30 high-resolution pictures of the Korean Peninsula every hour for weather observation [10, 42]; more than 60,000 new tracks are ingested by Spotify every day [24]; more than 720,000 hours of new video content is uploaded to YouTube every day [51]; and 95 million photos and videos are shared on Instagram per day [59].

Proximity search is a well-known classical problem of finding data close (or similar) to a query. One popular proximity search is **k -Nearest Neighbor (k NN)** search, which aims to find the k data points closest to the query point. In k NN search, data are typically represented as a set of points in a high-dimensional space, where the proximity properties of the original data are maintained to enable efficient distance computation. One way to perform k NN search is to compute the distances to all data points from the query point and select the k closest ones exactly. With large datasets, this exhaustive distance computation becomes impractical. To overcome this challenge, several recent studies [13, 17, 20, 25, 30, 36] have proposed indexing methods for fast and approximate k NN search.

However, the aforementioned studies do not consider the temporal information of the data, and therefore do not support **Time-Restricted k NN (TkNN)** search, such as “Which 5 movies released between 1980 and 1995 are most similar to Zootopia?” or “Which 10 photos that you took between January 2010 and May 2011 are most similar to the one you just took?” One way to handle TkNN queries using the above indexing methods is to perform k NN search on the entire dataset and filter the results to include only those within the time window. However, this method cannot guarantee that the number of search results is k and may even output nothing. Searching can continue until the desired number of results is obtained, but this approach may be time-consuming when the time window is short because most of the results have to be filtered out.

In this article, we propose **Multi-Level Block Indexing (MBI)**, a novel indexing method for efficient approximate TkNN search on time-accumulating data with timestamps. MBI aims to ensure efficient processing of any TkNN query, regardless of whether its time window is long or short, and to ensure efficient addition of new data that continuously occurs over time to the index. For this purpose, we devise an expandable hierarchical indexing structure where the data are divided into multiple blocks based on timestamps. Then, MBI efficiently processes queries by selecting blocks that match the query time window.

To make MBI more versatile, we also generalize TkNN to the **m -Attribute Filtering k NN (m -AkNN)** problem and extend MBI to support cases where the number m of filtering attributes is 2 or 3. While TkNN considers time as the sole filtering attribute, many real-world applications require filtering based on multiple numerical attributes. For example, medical datasets often contain high-dimensional vector representations (e.g., CT scan embeddings) along with patient characteristics such as age, height, and weight, which are essential for refining search results. Similarly, in retrieval-augmented generation models [34], document embeddings are stored with metadata like citation counts and publication years, enabling more precise document retrieval. To efficiently handle such queries, we enhance MBI with tree-based spatial partitioning structures such as quadrees and octrees, enabling scalable and efficient multi-attribute filtering in k NN search. Our contributions are summarized as follows:

- *Method.* We propose MBI, a new indexing algorithm for efficient approximate TkNN search on time-accumulating data with timestamps.

Table 1. Table of Symbols

Symbol	Definition
$\mathcal{T}$	Set of timestamps
$\mathcal{D}$	Spatio-temporal database
(v, t)	Timestamped vector where $v \in \mathbb{R}^d$ is a vector and $t \in \mathcal{T}$ is a timestamp
d	Spatial dimension of $\mathcal{D}$
$\mathcal{D}[t_a : t_b]$	Subset of $\mathcal{D}$ where every timestamped vector (v, t) satisfies $t_a \leq t < t_b$
S_L	Leaf block size in MBI
τ	Threshold for selecting blocks in MBI
$\mathcal{B}_i = (\mathcal{D}_i, \mathcal{G}_i)$	Block of index i in MBI where $\mathcal{D}_i$ is its timestamped vector set and $\mathcal{G}_i$ is its graph-based index for TkNN queries
$\mathcal{B}_i.t_s, \mathcal{B}_i.t_e$	The earliest and the latest timestamp of vectors in $\mathcal{B}_i$
$q = (w, k, t_s, t_e)$	TkNN query where w is a vector, k is the number of query results, and t_s and t_e are the start and the end timestamps of the query time window
m	Number of attribute constraints in m -AkNN query
$(v, \mathbf{a})$	Vector $v \in \mathbb{R}^d$ with attribute values $\mathbf{a} = (a_1, \dots, a_m)$
$\hat{\mathbf{a}}, \bar{\mathbf{a}}$	Lower and upper bounds of attribute constraints in m -AkNN query ($\hat{\mathbf{a}} = (\hat{a}_1, \dots, \hat{a}_m), \bar{\mathbf{a}} = (\bar{a}_1, \dots, \bar{a}_m) \in \mathbb{R}^m$)
$\mathcal{D}[\hat{\mathbf{a}} : \bar{\mathbf{a}}]$	Subset of $\mathcal{D}$ satisfying the attribute constraint $\hat{\mathbf{a}} : \bar{\mathbf{a}}$
$q = (w, k, \hat{\mathbf{a}}, \bar{\mathbf{a}})$	m -AkNN query where w is a vector, k is the number of query results

- *Analysis.* We theoretically analyze the efficiency of MBI in terms of index size, indexing time complexity and query time complexity.
- *Generalization.* We generalize TkNN to m -AkNN and extend MBI with quadrees and octrees to support cases where $m = 2$ or 3 .
- *Experiments.* The experimental results show that query processing of MBI is up to 10.88 times faster than that of the simple methods, and the data insertion time scales logarithmically with the total size of the data.

The codes and datasets used in this article are available at <https://github.com/kjsoo-1010/mbi2025>. This article extends our previous work [22]. Frequently used symbols are listed in Table 1.

2 Related Work

2.1 Approximate k NN Search

As exact k NN search can be computationally intensive, especially when dealing with high-dimensional datasets, various approximate k NN search methods have been studied and developed to overcome this challenge. These methods aim to find an approximation of the k NNs of a given query point, while minimizing the computational cost. One key idea behind these methods is to build a data structure that allows efficient and fast search of the k NNs.

We categorize these methods based on their underlying structures and approaches: tree-based, graph-based, hashing-based, and quantization-based methods. Tree-based methods including KD-Tree [14], R*-Tree [6], Balltree [43], Cover trees [8], RPTree [12], and VP-Tree [61] use recursive partitioning to organize the data. RPTree [60] and FLANN [39] utilize multiple trees to improve the search efficiency and reduce the effects of data distribution or randomization on the accuracy of the search results. Graph-based methods connect nearby data points using various types of graph structures, such as k NN graphs [44], Delaunay graphs [4], relative neighborhood graphs [52], and minimum spanning trees. Examples of the graph-based methods include NSW [35], HNSW [36], NGT [25], NNDescend [13], OLGraph [63], FANNG [23], and EFANNA [15].

A comprehensive survey that covers various graph-based methods is available in [57]. Hashing-based methods, such as LSH [18] and PUFFINN [3], group close points together using locality-sensitive hash functions. Quantization-based methods, including IVFADC [29], QuickADC [1], and ScaNN [20], represent each data point as a low-dimensional, discretized vector, which allows for fast distance computations by comparing the discretized vectors. Meanwhile, some studies improve the search performance through modern hardware support. SPANN [9] and DiskANN [26] use external memory to address the issue of memory shortage that can occur when processing large datasets. Faiss [28], PQT [58], and SONG [62] exploit GPUs to accelerate the search process. QuickerADC [2] increases the search speed of QuickADC by using SIMD instruction sets.

As such, a wide range of approximate k NN search methods have been developed, among which graph-based and quantization-based methods have demonstrated the state-of-the-art performance [7]. However, applying these methods directly to Tk NN search is challenging. By creating indices for the entire dataset without considering timestamps, existing approximate k NN methods are hard to efficiently filter data that falls within a specific time window, which is exactly what Tk NN requires. Meanwhile, these methods can be adapted by continuing the search until k or more results satisfy the time condition. However, this approach leads to severe performance degradation when the query time window is narrow, as it must explore a disproportionately large portion of the data to meet the k results requirement. We discuss this modification further in Section 3.2.2. In this article, we propose a novel method that empowers these methods to efficiently perform Tk NN search, irrespective of the length of the query time window. Furthermore, we extend our approach to the more general m - Ak NN problem, where filtering is required across multiple attributes rather than just time. As in Tk NN, m - Ak NN also faces the same fundamental challenge of efficiently retrieving k NNs while satisfying the given filtering conditions. We note that our method uses existing approximate k NN search methods as a module.

2.2 Tk NN Search

Tk NN search has been studied mainly in the fields of geoinformatics and databases with the purpose of performing queries on objects moving over time in 2D or 3D space. Tk NN search has been referred to as Spatio-Temporal k NN in existing literature, given that vectors represent spatial coordinates. A simple approach is to treat time as a new spatial axis and apply tree structures such as R^* -Tree or quadtree [32, 40, 46, 47, 49, 50, 54]. In this approach, Tk NN search can be performed by conducting the range search on the time axis and the k NN search on the remaining axes. Similarly, PPR-Tree [21, 32] is specifically designed for moving objects and links ephemeral R -Trees in a directed acyclic graph format over time for Tk NN search. DISTIL+ [38] is based on the quadtree structure and utilizes distributed clusters to process large datasets. However, none of the aforementioned methods are well-suited for Tk NN search in high-dimensional data, as the underlying tree structures become inefficient due to the curse of dimensionality [37]. Consequently, it becomes inevitable to explore almost all vectors within the query time window when dealing with high-dimensional data, as most bounding boxes in the R -tree-based index tend to overlap heavily, greatly reducing pruning efficiency and causing excessive node accesses. Meanwhile, Tk NN search in high-dimensional data is becoming essential with the advancement of artificial intelligence as various types of data such as photos, music, and documents are represented as high-dimensional data [33, 53, 55]; this naturally applies to m - Ak NN as well. In this article, we propose a new method that performs approximate Tk NN and m - Ak NN search efficiently even in high-dimensional data.

3 Preliminaries

In this section, we first provide a formal definition of the TkNN search, the key problem we address in this article. We then introduce m -AkNN search problem as a generalized form of TkNN. Following that, we introduce two straightforward approaches and show their limitations.

3.1 Problem Definition

Before defining the TkNN search, let us first introduce several necessary terms and symbols. A timestamped vector (v, t) is a pair consisting of a vector $v \in \mathbb{R}^d$ and a timestamp $t \in \mathcal{T}$, where d is the dimension of the vector space and $\mathcal{T}$ is a set of timestamps. Any two timestamps in $\mathcal{T}$ are comparable; for $t_a, t_b \in \mathcal{T}$, $t_a < t_b$ indicates that t_a precedes t_b . A database $\mathcal{D} \subseteq \mathbb{R}^d \times \mathcal{T}$ is a set of timestamped vectors. For $t_a, t_b \in \mathcal{T}$ such that $t_a < t_b$, we denote by $\mathcal{D}[t_a : t_b]$ a subset of $\mathcal{D}$ whose vectors have timestamps between t_a and t_b , i.e., $\mathcal{D}[t_a : t_b] = \{(v, t) \in \mathcal{D} \mid t_a \leq t < t_b\}$. The proximity of two vectors is defined as a distance function $\sigma : \mathbb{R}^d \times \mathbb{R}^d \rightarrow \mathbb{R}^+$, and we denote by $\sigma(u, v)$ the distance between vectors u and v . Any distance measure including the euclidean distance can be used for σ depending on datasets and applications. Then, we define a TkNN query as follows.

Definition 1. A TkNN query $q = (w, k, t_s, t_e)$ on database $\mathcal{D}$ is to identify a subset A of $\mathcal{D}$ such that:

$$A = \arg \min_{X \subseteq \mathcal{D}[t_s : t_e], |X|=k} \sum_{(v, t) \in X} \sigma(w, v),$$

where $w \in \mathbb{R}^d$ is the query vector, k is the number of timestamped vectors to identify, and $t_s, t_e \in \mathcal{T}$ are the start and end timestamps of the query time window.

For the sake of simplicity, we assume that all vectors have distinct timestamps. Even if there are vectors with identical timestamps, as in real-world datasets, our approach can be easily adapted. For vectors sharing the same timestamp, we arbitrarily assign an order. During the query process, we set the query range from the earliest ordered vector with the start timestamp to the last ordered vector with the end timestamp. This ensures that all vectors with the same timestamp are included in the search when the timestamp falls within the query range, and none are included otherwise.

A TkNN query applies a range search on the timestamp while performing a k NN search on the query vector. Here, the timestamp is the only attribute subjected to the range search. We extend this to multiple attributes—such as age, height, and weight in medical data—and define the m -AkNN search. An m -AkNN query is performed on a database $\mathcal{D} \subseteq \mathbb{R}^d \times \mathbb{R}^m$ of attributed vectors, where each vector $v \in \mathbb{R}^d$ is associated with m attributes $\mathbf{a} = (a_1, \dots, a_m) \in \mathbb{R}^m$. Since attributes are real-valued, they are naturally comparable, just like timestamps. For $\hat{\mathbf{a}} = (\hat{a}_1, \dots, \hat{a}_m), \tilde{\mathbf{a}} = (\tilde{a}_1, \dots, \tilde{a}_m) \in \mathbb{R}^m$ such that $\hat{a}_i < \tilde{a}_i$ for $i = 1, \dots, m$, we denote by $\mathcal{D}[\hat{\mathbf{a}} : \tilde{\mathbf{a}}]$ a subset of $\mathcal{D}$ whose vectors have attributes between $\hat{\mathbf{a}}$ and $\tilde{\mathbf{a}}$, i.e., $\mathcal{D}[\hat{\mathbf{a}} : \tilde{\mathbf{a}}] = \{(v, \mathbf{a}) \in \mathcal{D} \mid \hat{a}_i \leq a_i < \tilde{a}_i \forall i \in \{1, \dots, m\}\}$. Then, an m -AkNN query is defined as follows.

Definition 2. An m -AkNN query $q = (w, k, \hat{\mathbf{a}}, \tilde{\mathbf{a}})$ on database $\mathcal{D}$ is to identify a subset A of $\mathcal{D}$ such that:

$$A = \arg \min_{X \subseteq \mathcal{D}[\hat{\mathbf{a}} : \tilde{\mathbf{a}}], |X|=k} \sum_{(v, \mathbf{a}) \in X} \sigma(w, v),$$

where $w \in \mathbb{R}^d$ is the query vector, k is the number of vectors to retrieve, and $\hat{\mathbf{a}}, \tilde{\mathbf{a}} \in \mathbb{R}^m$ specify the lower and upper bounds on the attribute values that a result must satisfy, respectively.

Algorithm 1: BSBF Query Process

Input: A TkNN query $q = (w, k, t_s, t_e)$, a database $\mathcal{D}$ in order of increasing timestamp, a distance function σ

Output: TkNNs of q

```

1  $\mathcal{D}[t_s : t_e] \leftarrow \text{BinarySearch}(t_s, t_e, \mathcal{D})$ 
2 return  $\text{BruteForceKNN}(w, k, \sigma, \mathcal{D}[t_s : t_e])$ 

```

Algorithm 2: AFBF Query Process

Input: A query $q = (w, k, \hat{a}, \check{a})$, a database $\mathcal{D}$, a distance function σ

Output: m -AkNNs of q

```

1  $\mathcal{D}[\hat{a} : \check{a}] \leftarrow \text{BruteForceFiltering}(\hat{a}, \check{a}, \mathcal{D})$ 
2 return  $\text{BruteForceKNN}(w, k, \sigma, \mathcal{D}[\hat{a} : \check{a}])$ 

```

For simplicity, we assume that the attributes of each attributed vector are distinct from each other, as with timestamps in TkNN.

3.2 Two Simple Approaches for m -AkNN Search

We introduce two straightforward approaches to handle TkNN queries, extend them for m -AkNN queries, and discuss their limitations.

3.2.1 Binary Search and Brute-Force (BSBF). BSBF is a method that combines binary search and brute-force methods for TkNN search. Algorithm 1 shows the pseudocode for BSBF's query process. BSBF sorts all timestamped vectors in database $\mathcal{D}$ in order of increasing timestamp and uses the sorted database as its index structure. Given a TkNN query $q = (w, k, t_s, t_e)$, BSBF identifies $\mathcal{D}[t_s : t_e]$ by performing binary search process and finds the k vectors in $\mathcal{D}[t_s : t_e]$ that are closest to w using a brute-force method. The query process of BSBF is very fast when the query time window is short. Let n and b be the sizes of $\mathcal{D}$ and $\mathcal{D}[t_s : t_e]$, respectively. The query process of BSBF requires $O(\log n)$ to identify $\mathcal{D}[t_s : t_e]$ and $O(bd \log k)$ to find k nearest points assuming a max-heap of size k is used for the brute-force method where d is the dimension of the vectors. This implies that when b is close to n , BSBF becomes severely inefficient because it must examine nearly all the timestamped vectors in $\mathcal{D}$.

The algorithm that applies BSBF to the m -AkNN problem is called **Attribute-Filtering Brute-Force (AFBF)**. The pseudocode of AFBF's query process is presented in Algorithm 2. Unlike TkNN, where a single attribute allows for efficient binary search, the presence of multiple attributes in m -AkNN makes binary search impractical. Therefore, AFBF replaces the binary search step with a brute-force method to identify data points that satisfy the attribute constraints. Given an m -AkNN query $q = (w, k, \hat{a}, \check{a})$, let n and b denote the sizes of $\mathcal{D}$ and $\mathcal{D}[\hat{a} : \check{a}]$, respectively. AFBF takes $O(mn)$ time to identify $\mathcal{D}[\hat{a} : \check{a}]$ and $O(bd \log k)$ time to perform brute-force k NN search within $\mathcal{D}[\hat{a} : \check{a}]$. In contrast, brute-force k NN search over the entire dataset $\mathcal{D}$ requires $O(nd \log k)$ time. Since $m \ll d$, AFBF remains highly efficient when $b \ll n$. However, if b approaches n , AFBF becomes inefficient, as in the case of BSBF.

3.2.2 Search and Filtering (SF). Another approach for handling TkNN queries is SF, which uses an existing approximate k NN method [25] with the following minor modification: it continues searching until it identifies k or more vectors within the query condition window. Algorithm 3 provides the pseudocode for handling a TkNN query in a proximity graph $\mathcal{G}$ using SF. Lines 12–15 are mainly different from the existing algorithm.

Algorithm 3: Graph-Based SF Query Process

Input: A TkNN query $q = (w, k, t_s, t_e)$, a graph $\mathcal{G}$ of a database $\mathcal{D}$, a distance function σ , a maximum candidate set size M_C , a parameter ϵ for controlling the search range

Output: Approximate TkNNs of q

```

1  $s \leftarrow$  a randomly sampled timestamped vector from  $\mathcal{D}$ 
2 Initialize a candidate set  $C \leftarrow \{s\}$ 
3 Initialize a visited set  $V \leftarrow \emptyset$ 
4 Initialize a result set  $R \leftarrow \emptyset$ 
5 while  $C \setminus V \neq \emptyset$  do
6    $p' = (v', t') \leftarrow \arg \min_{(v,t) \in C \setminus V} \sigma(v, w)$ 
7    $V \leftarrow V \cup \{p'\}$ 
8   if  $|R| < k$  then
9      $C \leftarrow C \cup \{ \text{neighbors}(v'', t'') \text{ of } p' \text{ in } \mathcal{G} \}$ 
10  else
11     $C \leftarrow C \cup \{ \text{neighbors}(v'', t'') \text{ of } p' \text{ in } \mathcal{G} \text{ such that } \sigma(v'', w) < \epsilon \cdot \max_{(v,t) \in R} \sigma(v, w) \}$ 
12  if  $t_s \leq t' < t_e$  then
13     $R \leftarrow R \cup \{p'\}$ 
14    if  $|R| > k$  then
15       $\text{update } R \text{ to retain } k \text{ nearest timestamped vectors to } w$ 
16  if  $|C| > M_C$  then
17     $\text{update } C \text{ to retain } M_C \text{ nearest timestamped vectors to } w$ 
18 return  $R$ 

```

The proximity graph $\mathcal{G}$ is built using any proximity-based indexing techniques like NNDescent to connect spatially close vertices, where each vertex corresponds to a vector in $\mathcal{D}$ without attributes. This algorithm starts at a random vertex s and traverses the graph towards the query vector w . During the traversal, it filters and builds a result set R from vectors that fall within the query time window. SF is effective when the query time window is wide enough to cover almost all vectors in the database $\mathcal{D}$. However, it becomes inefficient when the query time window is narrow. This is because a narrow query time window only includes a few vectors, and most of the vectors identified during the search do not make it into the results, which significantly expands the search area.

SF naturally extends to the m -AkNN problem. The modification only requires replacing the part that checks whether a candidate vector exists within the query time window (line 12 in Algorithm 3) with a condition that verifies whether the vector satisfies all attribute constraints of the m -AkNN query. Similar to the TkNN context, this modified SF becomes severely inefficient when the number of data points satisfying the attribute constraints, $\mathcal{D}[\tilde{a} : \tilde{a}]$, is small.

4 Proposed Method

In this section, we propose MBI, a new incremental indexing method¹ for TkNN queries that works efficiently for queries of any time windows. We also extend this approach to handle m -AkNN queries. The technical challenges in efficiently implementing MBI and our key ideas to resolve them are as follows:

- C1. How can we make the query process efficient regardless of the query time window's length?*
 When the query time window is long, BSBF is inefficient because it must check all vectors within the window. On the other hand, when the query time window is short, SF becomes

inefficient as it must expand the search area until the targeted number of vectors within the time window is detected. *To ensure the efficiency for queries of any time window, we divide the database into multiple blocks based on timestamps and organize them in a hierarchical structure* (Section 4.1). Accordingly, MBI operates like BSBF when the query time window is short, and like SF when the query time window is long.

- C2. *How can we efficiently add new vectors that occur over time to MBI?* Data containing timestamps usually accumulates continuously over time. Meanwhile, it's not straightforward to insert new data while maintaining the sophisticated hierarchical structure of MBI. *We enable efficient incremental construction of MBI by bottom-up block merging and postorder block numbering* (Section 4.2).
- C3. *How should we select a set of blocks to process the query most efficiently?* The query processing of MBI involves finding a set of time-disjoint blocks that covers the query time window, performing TkNN queries independently on each selected block, and then merging the results. The performance of the query processing depends on the selected set of blocks; if we only select small blocks, it becomes inefficient when the query time window is long, similar to BSBF, and if we only select large blocks, it becomes inefficient when the query time window is short, similar to SF. *We propose a top-down block selection method that efficiently finds a mixed set of both large and small blocks suitable for each query* (Section 4.3).
- C4. *How can we extend MBI to support m -AkNN queries when $m > 1$?* MBI employs a complete binary tree index to solve the TkNN problem, which corresponds to the case where $m = 1$. However, for $m > 1$, a complete 2^m -ary tree cannot be constructed unless the data forms a perfectly structured grid. *We extend MBI to a 2^m -ary tree indexing method based on count-based recursive space partitioning, to efficiently handle the m -AkNN problem when $m > 1$* (Section 4.4).

In the following subsections, we first introduce MBI for TkNN, which covers its overview, indexing, and query processing in Sections 4.1, 4.2, and 4.3, respectively. We then extend MBI to support m -AkNN in Section 4.4, where we discuss its overview and indexing in Section 4.4.1 and query processing in Section 4.4.2. Finally, we provide several theoretical analyses of MBI in Section 4.5.

4.1 Overview

MBI is a binary tree of blocks, each containing all vectors of a specific time window and an index to efficiently handle TkNN queries on these vectors. While any index structure for efficient k NN search can be used for the index, we employ one of the graph-based indexing methods, as they have shown the best search performance. Each block in MBI has the left and right children, each having half of the vectors in the parent block. The left child has the half with earlier timestamps, while the right child does the half with later timestamps. The root block holds all vectors, and each leaf block holds S_L or fewer vectors. Figure 1 shows an example of MBI. The database $\mathcal{D} = \mathcal{D}[t_0 : t_{15}]$ contains 16 vectors, each with a timestamp from t_0 to t_{15} . If $i < j$, then $t_i < t_j$. The leaf size S_L is 4. The vectors are color-coded based on the order of their timestamps.

MBI enables efficient query processing by searching only in a few selected blocks that overlap with the query's time window. For example, in Figure 1, a query with time window ($t_s = t_5, t_e = t_7$) can be efficiently processed by considering only block $\mathcal{B}_1$ whose vector set is $\mathcal{D}[t_4 : t_7]$. Similarly, a query with time window ($t_s = t_1, t_e = t_{15}$) can be efficiently processed with $\mathcal{B}_6$. Meanwhile, a query with time window ($t_s = t_4, t_e = t_{14}$) can be processed with different sets of blocks: $\{\mathcal{B}_1, \mathcal{B}_5\}$, $\{\mathcal{B}_1, \mathcal{B}_3, \mathcal{B}_4\}$, or $\{\mathcal{B}_6\}$. We will explain our strategy for finding a set of blocks for efficient query processing in Section 4.3.

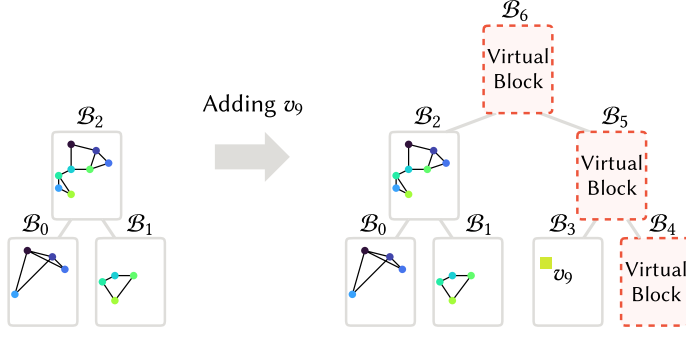


Fig. 2. An example of adding a new vector into MBI whose leaf blocks are full.

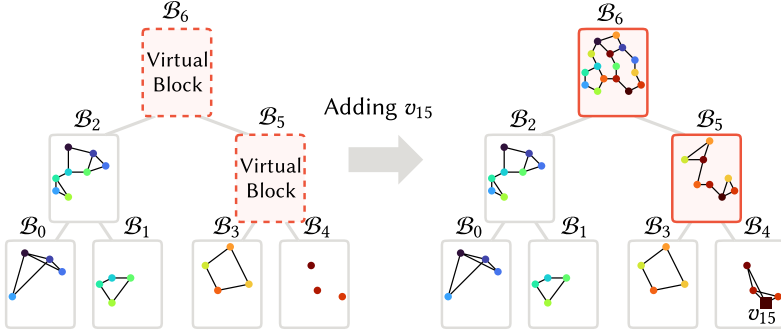


Fig. 3. An example of MBI's bottom-up block merging process.

Algorithm 4: MBI's Vector Insertion

Input: MBI, a timestamped vector (v, t)

- 1 Let i be the index of the first non-full leaf block in MBI
 - 2 Let $\mathcal{B}_i = (\mathcal{D}_i, \mathcal{G}_i)$ be the block at index i where $\mathcal{D}_i$ is the vector set and $\mathcal{G}_i$ is the graph-based k NN index, which is initially empty.
 - 3 $\mathcal{D}_i \leftarrow \mathcal{D}_i \cup \{(v, t)\}$
 - 4 **if** $|\mathcal{D}_i| = S_L$ **then**
 - 5 $\mathcal{G}_i \leftarrow \text{BuildKNNIndex}(\mathcal{D}_i)$
 - 6 Let j be the number of leaf blocks in MBI
 - 7 $h \leftarrow 1$
 - 8 **while** j is even **do**
 - 9 $\mathcal{D}_{i+1} \leftarrow \mathcal{D}_{i+1-2^h} \cup \mathcal{D}_i$
 - 10 $\mathcal{G}_{i+1} \leftarrow \text{BuildKNNIndex}(\mathcal{D}_{i+1})$
 - 11 $\mathcal{B}_{i+1} \leftarrow (\mathcal{D}_{i+1}, \mathcal{G}_{i+1})$
 - 12 $i \leftarrow i + 1$
 - 13 $j \leftarrow j/2$
 - 14 $h \leftarrow h + 1$
-

Algorithm 5: MBI Query Process

Input: A TkNN query $q = (w, k, t_s, t_e)$, MBI, a distance function σ
Output: An approximate TkNNs of q

```

1 Let  $\mathcal{B}_r$  be the root block of MBI
2  $R \leftarrow \emptyset$ 
3  $S \leftarrow \text{BlockSelection}(\mathcal{B}_r, t_s, t_e)$ 
4 foreach  $\mathcal{B}' \in S$  do
5   if  $\mathcal{B}'$  is a non-full leaf block then
6      $R \leftarrow R \cup \text{BSBFQuery}(w, k, t_s, t_e, \mathcal{B}', \sigma)$  // Algorithm 1
7   else
8      $R \leftarrow R \cup \text{SFQuery}(w, k, t_s, t_e, \mathcal{B}', \sigma)$  // Algorithm 3
9    $R \leftarrow k$  neighbors nearest to  $w$  in  $R$ 
10 return  $R$ 
11 Function  $\text{BlockSelection}(\mathcal{B}_c, t_s, t_e)$ :
12   Let  $\mathcal{B}_c.t_s$  and  $\mathcal{B}_c.t_e$  be the earliest and the exclusive upper timestamps of vectors in  $\mathcal{B}_c$ ,
    respectively.
13    $r_o \leftarrow \frac{\max(0, \min(\mathcal{B}_c.t_e, t_e) - \max(\mathcal{B}_c.t_s, t_s))}{\mathcal{B}_c.t_e - \mathcal{B}_c.t_s}$ 
14   if  $r_o = 0$  then
15     return  $\emptyset$ 
16   else if  $\mathcal{B}_c$  is a leaf block or  $r_o > \tau$  then
17     return  $\{\mathcal{B}_c\}$ 
18   else
19     Let  $c$  and  $h$  be the index and the height of  $\mathcal{B}_c$ , respectively.
20     return  $\text{BlockSelection}(\mathcal{B}_{c-2^h}, t_s, t_e) \cup \text{BlockSelection}(\mathcal{B}_{c-1}, t_s, t_e)$ 

```

$\mathcal{D}_i$ and we build a k NN index $\mathcal{G}_{i+1}$ where h is the height of the parent block in the tree. This process is repeated in the parent block. Note that the sibling block of $\mathcal{B}_i$ is at index $i + 1 - 2^h$, as blocks are numbered sequentially corresponding to a postorder traversal of MBI.

Parallelization of MBI. The bottom-up block merging process is easily parallelizable because it builds each block independently. For example, when v_{15} is added, MBI independently identifies the vector sets of $\mathcal{B}_4$, $\mathcal{B}_5$, and $\mathcal{B}_6$ by combining the vector sets of $\mathcal{B}_2$, $\mathcal{B}_3$, and $\mathcal{B}_4$, then, the k NN indices $\mathcal{G}_4$, $\mathcal{G}_5$, and $\mathcal{G}_6$ are built in parallel.

4.3 Query Processing on MBI

Algorithm 5 lists the pseudocode of query processing on MBI. Given a TkNN query $q = (w, k, t_s, t_e)$, we first find a set of blocks, referred to as a *search block set*, that covers all vectors whose timestamps are between t_s and t_e (line 3). We then perform TkNN search for each block in the search block set and combine the results to derive the final TkNN result (lines 4–7). To choose the best search block set among all possible ones, we propose a *top-down block selection* approach. Let $\mathcal{B}_c.t_s$ and $\mathcal{B}_c.t_e$ be the earliest and the exclusive upper timestamp of vectors in the *current block* $\mathcal{B}_c$, respectively. We define the *overlap ratio* $r_o(q, \mathcal{B}_c)$ of the query q in $\mathcal{B}_c$ as follows:

$$r_o(q, \mathcal{B}_c) = \frac{\max(0, \min(\mathcal{B}_c.t_e, t_e) - \max(\mathcal{B}_c.t_s, t_s))}{\mathcal{B}_c.t_e - \mathcal{B}_c.t_s}.$$

Then, the top-down block selection starts from the root block and proceeds according to the following three cases (lines 9–17):

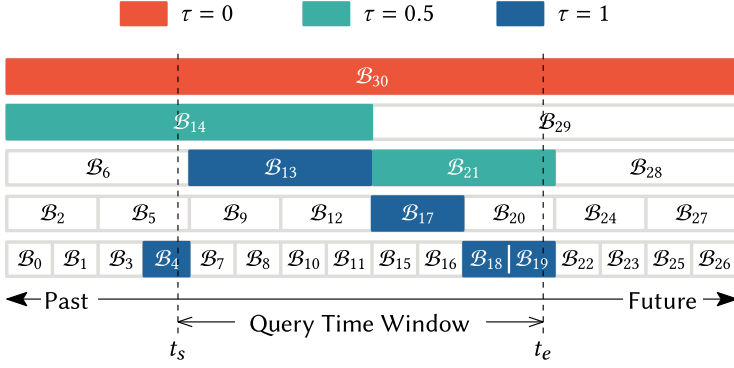


Fig. 4. Examples of search block sets when τ is 0, 0.5, and 1, respectively.

Case 1. If $r_o(q, \mathcal{B}_c)$ is 0, $\mathcal{B}_c$ is not included in the search block set.

Case 2. If $\mathcal{B}_c$ is a leaf block or the overlap ratio $r_o(q, \mathcal{B}_c)$ exceeds a threshold τ , the block is included in the search block set.

Case 3. If $\mathcal{B}_c$ is a non-leaf block and $r_o(q, \mathcal{B}_c)$ is less than τ , the block is not included in the search block set, and this process is repeated for its child blocks.

The value of τ determines the composition of the search block set. As τ approaches 0, blocks closer to the root are more likely to be selected, whereas as τ approaches 1, the selection criteria for blocks become stricter, increasing the probability of selecting leaf blocks. Figure 4 illustrates examples of search block sets corresponding to different τ values. In this example, when τ is 0, 0.5, and 1, the search block sets are $\{\mathcal{B}_{30}\}$, $\{\mathcal{B}_{14}, \mathcal{B}_{21}\}$, and $\{\mathcal{B}_4, \mathcal{B}_{13}, \mathcal{B}_{17}, \mathcal{B}_{18}, \mathcal{B}_{19}\}$, respectively. A τ value that is too low makes the query process performed only on the root block, significantly degrading the performance when the query time window is short as in the case of SF. On the other hand, a τ value that is too high makes the query process performed on too many blocks, increasing the overhead, especially when the query time window is long. The optimal value of τ varies according to the dataset. In Section 4.5.3, we prove that the number of blocks in which MBI performs a query is at most two if $\tau \leq 0.5$. Also, in Section 5.4, we experimentally show how τ affects the search performance.

In the incremental process of adding vectors to MBI, it may not always form a complete tree as in the left tree of Figure 3. In such cases, to conduct the search as described above, we complete the tree with virtual blocks whose time window extends from $-\infty$ to ∞ . Throughout the top-down block selection process, these virtual blocks always fall into case 3 due to their infinite time window size. Consequently, they are never included in the search block set, providing opportunities for their descendants to be potentially included instead.

4.4 Extension to m -AkNN

This section proposes an extension of MBI to efficiently handle the m -AkNN problem, which generalizes the TkNN problem by extending it from a single attribute (timestamp) to multiple attributes. In the case of the TkNN problem, where there is only a single attribute, MBI organizes blocks in a binary tree structure. Extending this concept to cases where $m \geq 2$, MBI structures blocks as a 2^m -ary tree based on multiple attributes.

One key difference between the TkNN and m -AkNN problems is that in TkNN, data are assumed to arrive in increasing order of timestamps, which is a natural scenario in many applications, whereas in m -AkNN, attribute values are independent of the insertion order. In TkNN, MBI can

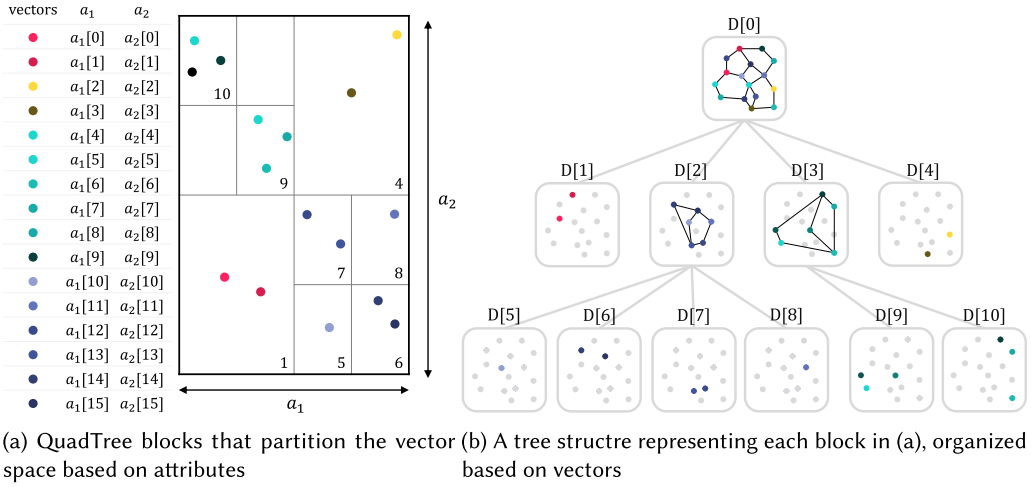


Fig. 5. An example of the quadtree-based index structure of MBI when $m = 2$.

efficiently maintain the index structure by always inserting new data into the last block and forming blocks via bottom-up merging. However, in m -AkNN, the same strategy cannot be applied. Thus, we assume that data insertion and deletion are not supported in the m -AkNN setting.

For $m = 1$, MBI creates blocks by dividing the data in the order of the attribute, ensuring that each block contains an equal number of data points. A bottom-up approach is then applied, where lower-level blocks are recursively merged in pairs to form higher-level blocks. Applying the same approach to cases where $m \geq 2$ requires partitioning the data space into a uniform grid. However, achieving uniformly distributed blocks is impractical for real-world datasets. Accordingly, the majority of blocks would remain empty, leading to severe inefficiencies. To address this issue, we employ a top-down hierarchical block partitioning strategy, where $m = 2$ and $m = 3$ result in a quadtree and an octree, respectively. Instead of enforcing a rigid grid structure, our approach adaptively divides the data space based on density, allowing the tree to grow deeper in highly populated areas while remaining shallow in sparsely populated ones.

Figure 5 illustrates an example of the quadtree-based index structure of MBI when $m = 2$. Figure 5(a) illustrates a database $\mathcal{D}$ and its vectors mapped to spatial coordinates based on their attribute values a_1 and a_2 , demonstrating a quadtree-based block partitioning. The i th vector has attribute values $(a_1[i], a_2[i])$. Vectors are color-coded according to their corresponding blocks, assuming that the attribute value $a_m[i]$ is independent of the vector index i . The leaf size S_L is 5. Figure 5(b) illustrates the database $\mathcal{D}$ from a vector-based perspective, presenting each block in Figure 5(a) as a hierarchical tree structure. It also visualizes the vectors contained in each block along with their corresponding k NN graph.

4.4.1 Indexing. The detailed process of how MBI indexes for m -AkNN when $m \geq 2$ using the top-down strategy is listed in Algorithm 6. MBI defines the range of the root block using the minimum and maximum values of the attributes to be filtered. At each level, the tree recursively partitions the parent block's range to generate child blocks. Each block is defined to contain all data points that satisfy the attribute range. That is, the root block encompasses the entire dataset. When the dataset is input into the root block (line 3), it is recursively partitioned until all leaf blocks contain at most S_L points (lines 11 and 12). During partitioning, the existing data points are

Algorithm 6: MBI Indexing Algorithm for m -AkNN

Input: MBI, Dataset D

- 1 Let $\mathcal{B}_i = (\mathcal{D}_i, \mathcal{G}_i)$ be the block at index i where $\mathcal{D}_i$ is the vector set and $\mathcal{G}_i$ is the graph-based k NN index, which is initially empty.
- 2 Let $\mathcal{B}_0$ be the root block of MBI
- 3 InsertAndSubdivide($\mathcal{N}_0$)
- 4 **Function** InsertAndSubdivide($\mathcal{B}_i$):
- 5 **if** $|\mathcal{D}_i| > S_L$ **then**
- 6 $\mathcal{G}_i \leftarrow \text{BuildKNNIndex}(\mathcal{D}_i)$
- 7 Subdivide $\mathcal{B}_i$ into 2^m child blocks by splitting each attribute at its midpoint
- 8 **foreach** vector $v \in \mathcal{D}_i$ **do**
- 9 Determine the appropriate child node $\mathcal{D}_c$ for v based on its m attributes
- 10 Add v to $\mathcal{D}_c$
- 11 **foreach** child block $\mathcal{B}_c$ of $\mathcal{B}_i$ **do**
- 12 InsertAndSubdivide($\mathcal{B}_c$)

preserved and reassigned to the child blocks (lines 8–10). Blocks containing more than S_L indices construct a k NN graph using NNDescent [13] (line 6).

4.4.2 Query Process. The query processing for m -AkNN adopts a similar top-down block selection strategy to TkNN to efficiently identify the blocks to be searched. The key difference lies in the consideration of multiple attributes when calculating the overlap ratio r_o between a block and the query, as well as the use of AFBF (Algorithm 2) instead of BSBF (Algorithm 1), which are mentioned in Section 3.2.1. Let $\mathcal{B}_a.\dot{a}_i$ and $\mathcal{B}_a.\ddot{a}_i$ be the lower and upper bounds, respectively, of the i th attribute of block $\mathcal{B}_a$. Given a m -AkNN query $q = (w, k, \dot{a}, \ddot{a})$, the overlap ratio r_o is then calculated as follows:

$$r_o(q, \mathcal{B}_a) = \prod_{i=1}^m \frac{\max(0, \min(\mathcal{B}_a.\ddot{a}_i, \ddot{a}_i) - \max(\mathcal{B}_a.\dot{a}_i, \dot{a}_i))}{\mathcal{B}_a.\ddot{a}_i - \mathcal{B}_a.\dot{a}_i}.$$

Algorithm 7 shows the pseudocode for the MBI query process for the m -AkNN problem. The changes from Algorithm 5, which describes the MBI query process for TkNN, are highlighted in blue.

Figure 6 illustrates examples of search block sets in MBI with a quadtree structure for $m = 2$ during the query process, showing different selected blocks for τ values of 0, 0.5, and 1. Similar to Figure 4, when τ approaches 0, high-level blocks, closer to the root block, are more likely to be selected, whereas when τ approaches 1, leaf blocks are more likely to be selected. In this example, when τ is 0, 0.5, and 1, the search block sets are $\{\mathcal{B}_0\}$, $\{\mathcal{B}_3, \mathcal{B}_7, \mathcal{B}_8, \mathcal{B}_{11}, \mathcal{B}_{13}, \mathcal{B}_{15}\}$, and $\{\mathcal{B}_3, \mathcal{B}_7, \mathcal{B}_8, \mathcal{B}_{13}, \mathcal{B}_{29}, \mathcal{B}_{30}\}$, respectively. In this case, when $\tau = 0.5$, $\mathcal{B}_7$ is included in the search block, and when $\tau = 1.0$, $\mathcal{B}_3, \mathcal{B}_7, \mathcal{B}_8$, and $\mathcal{B}_{11}$ are also included, despite not satisfying the τ threshold, since they are leaf blocks.

4.5 Analysis

This section provides analytical results of MBI. The indexing methods employed for each block significantly influence the analytical results. Let $\Phi(n)$ be the time complexity and $\Psi(n)$ the index size of the indexing method for n vectors.

Algorithm 7: MBI Query Process for m -AkNN

Input: A m -AkNN query $q = (w, k, \hat{a}, \ddot{a})$, MBI, a distance function σ
Output: An approximate m -AkNNs of q

- 1 Let $\mathcal{B}_0$ be the root block of the MBI
- 2 $R \leftarrow \emptyset$
- 3 $S \leftarrow \text{BlockSelection}(\mathcal{B}_0, \hat{a}, \ddot{a})$
- 4 **foreach** $\mathcal{B}' \in S$ **do**
 - 5 **if** $\mathcal{B}'$ is a non-full leaf block **then**
 - 6 $R \leftarrow R \cup \text{AFBFQuery}(w, k, \hat{a}, \ddot{a}, \mathcal{B}', \sigma)$ // Algorithm 2
 - 7 **else**
 - 8 $R \leftarrow R \cup \text{SFQuery}(w, k, \hat{a}, \ddot{a}, \mathcal{B}', \sigma)$ // Algorithm 3
 - 9 $R \leftarrow k$ neighbors nearest to w in R
- 10 **return** R
- 11 **Function** $\text{BlockSelection}(\mathcal{B}_c, \hat{a}, \ddot{a})$:
 - 12 Let $\mathcal{B}_c$'s attribute boundaries be defined by $\mathcal{B}_c.\hat{a}_i$ and $\mathcal{B}_c.\ddot{a}_i$ for each attribute
 - 13 $r_o(q, \mathcal{B}_c) \leftarrow \prod_{i=1}^m \frac{\max(0, \min(\mathcal{B}_c.\ddot{a}_i, \hat{a}_i) - \max(\mathcal{B}_c.\hat{a}_i, \ddot{a}_i))}{\mathcal{B}_c.\ddot{a}_i - \mathcal{B}_c.\hat{a}_i}$
 - 14 **if** $r_o = 0$ **then**
 - 15 **return** $\emptyset$
 - 16 **else if** $\mathcal{B}_c$ is a leaf block or $r_o > \tau$ **then**
 - 17 **return** $\{\mathcal{B}_c\}$
 - 18 **else**
 - 19 **return** $\text{BlockSelection}(\mathcal{B}_{c+1}, \hat{a}, \ddot{a}) \cup \dots \cup \text{BlockSelection}(\mathcal{B}_{c+2^m}, \hat{a}, \ddot{a})$

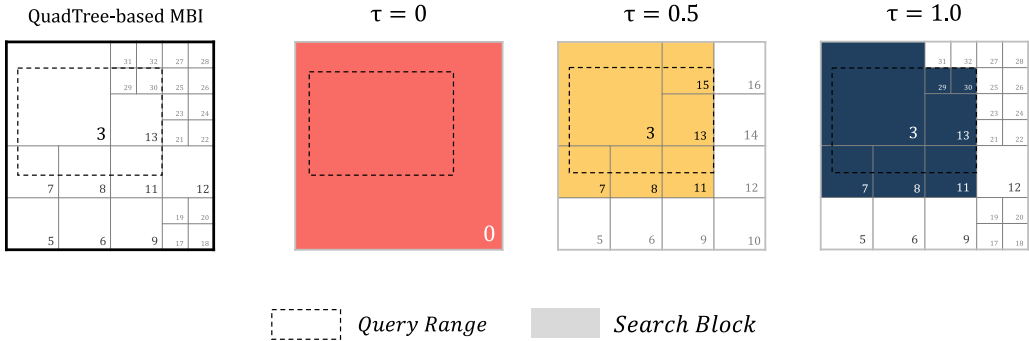


Fig. 6. Examples of search block sets with a quadtree structure for $m = 2$ when τ is 0, 0.5, and 1, respectively.

4.5.1 Index Size of MBI.

Index Size of MBI for TkNN. The total index size of MBI for TkNN is the sum of each block:

$$\sum_{i=0}^{\log \frac{|\mathcal{D}|}{S_L}} 2^i \Psi(|\mathcal{D}|/2^i),$$

where $|\mathcal{D}|$ is the number of vectors in database $\mathcal{D}$, and S_L is the leaf block size. The index size of a graph-based k NN index for n vectors is $O(nk')$, where k' is the average number of neighbors in the graph. Meanwhile, k' is usually set to be a constant value, so the required space can be seen as $O(n)$. Thus, indexing each block with a graph-based k NN index in MBI results in the following

total index size:

$$\sum_{i=0}^{\log \frac{|\mathcal{D}|}{S_L}} 2^i O(|\mathcal{D}|/2^i) = \sum_{i=0}^{\log \frac{|\mathcal{D}|}{S_L}} O(|\mathcal{D}|) = O(|\mathcal{D}| \log |\mathcal{D}|).$$

Index Size of MBI for m -AkNN. Even for m -AkNN, MBI requires the same index size of $O(|\mathcal{D}| \log |\mathcal{D}|)$. The maximum height of the tree formed by the blocks is $\log |\mathcal{D}|$. This is due to the fact that MBI divides the data in half for each attribute. The worst-case scenario occurs when the data are identically split by each attribute, resulting in the data being divided in half. Accordingly, if the number of data points in a block is N , the block's child blocks can have at most $N/2$ data points. Meanwhile, summing the number of data points across all levels of the tree results in a total that is less than or equal to $|\mathcal{D}|$. As the index size is proportional to the number of data points in the tree, it becomes $O(|\mathcal{D}| \log |\mathcal{D}|)$.

4.5.2 Indexing Time Complexity of MBI.

Indexing Time of MBI for TkNN. Since MBI indexes each block independently, the total indexing time is the sum of the time required to build each block, as follows:

$$\sum_{i=0}^{\log \frac{|\mathcal{D}|}{S_L}} 2^i \Phi\left(\frac{|\mathcal{D}|}{2^i}\right).$$

NNDescent, an approximate k NN graph build algorithm, is known to require $O(n^{1.14})$ time to index n vectors by empirical analysis [13, 48]. Indexing each block as a k NN graph using NNDescent, the total index time is as follows:

$$\sum_{i=0}^{\log \frac{|\mathcal{D}|}{S_L}} 2^i O\left(\left(\frac{|\mathcal{D}|}{2^i}\right)^{1.14}\right) = O\left(|\mathcal{D}|^{1.14} \sum_{i=0}^{\log \frac{|\mathcal{D}|}{S_L}} 2^{-0.14i}\right).$$

Since $2^{-0.14i}$ forms a geometric series with a common ratio of $2^{-0.14} < 1$, its sum converges to a constant. Accordingly, the total indexing time complexity is $O(|\mathcal{D}|^{1.14})$. The average time required to add a single vector to MBI, i.e., the amortized time complexity of inserting $|\mathcal{D}|$ th vector, is then as follows:

$$\frac{1}{|\mathcal{D}|} \times O(|\mathcal{D}|^{1.14}) = O(|\mathcal{D}|^{0.14}).$$

Note that $x^{0.14}$ grows sufficiently slowly; for example, $x^{0.14} < \log x$ when $2.17 < x < 1.6 \times 10^{11}$.

Indexing Time of MBI for m -AkNN. As in the case of TkNN, the total indexing time complexity of MBI for m -AkNN is the sum of the indexing times of all blocks. Assuming that the data are evenly distributed, the indexing time complexity is formulated as follows:

$$\sum_{i=0}^{\log_2 m \frac{|\mathcal{D}|}{S_L}} 2^{mi} \cdot \Phi\left(\frac{|\mathcal{D}|}{2^{mi}}\right),$$

where $\Phi(n) = O(n^{1.14})$ represents the time complexity of the NNDescent algorithm. Thus, the time complexity becomes:

$$\sum_{i=0}^{\log_2 m \frac{|\mathcal{D}|}{S_L}} 2^{mi} \cdot O\left(\left(\frac{|\mathcal{D}|}{2^{mi}}\right)^{1.14}\right) = O\left(|\mathcal{D}|^{1.14} \sum_{i=0}^{\log_2 m \frac{|\mathcal{D}|}{S_L}} 2^{-0.14mi}\right) = O(|\mathcal{D}|^{1.14}),$$

as $0 < 2^{-0.14m} < 1$ for $m \geq 1$.

We now analyze the worst-case indexing time in the general case where data are not evenly distributed. We define the level of a block as the number of edges on the path from the root block to that block, with the root block itself being at level 0. For simplicity, we denote by $\mathcal{B}_{\text{index}}^{\text{level}}$ a block in MBI. For example, the root block is $\mathcal{B}_0^0$, and the size $|\mathcal{B}_0^0|$ is $|\mathcal{D}|$. Then, the time required to construct all level j blocks is:

$$\sum_{i=0}^{2^{mj}-1} O(|\mathcal{B}_i^j|^{1.14}),$$

where the level j blocks satisfy two conditions: (1) the total sum of level j block sizes is at most $|\mathcal{D}|$, i.e., $\sum_{i=0}^{2^{mj}-1} |\mathcal{B}_i^j| \leq |\mathcal{D}|$, and (2) the maximum size of a single block is at most $|\mathcal{D}|/2^j$, i.e., $|\mathcal{B}_i^j| \leq |\mathcal{D}|/2^j$ for all $i = 0, \dots, 2^{mj} - 1$. Since the function $x^{1.14}$ is convex, the summation attains its maximum value when the block sizes are most unbalanced. That is, when 2^j blocks have a size of $|\mathcal{D}|/2^j$ while the remaining blocks have size 0, the maximum value is achieved: $2^j \times O((|\mathcal{D}|/2^j)^{1.14}) = O(|\mathcal{D}|^{1.14} \times 2^{-0.14j})$. Finally, the total indexing time across all levels is then:

$$\sum_{j=0}^{\log \frac{|\mathcal{D}|}{S_L}} O(|\mathcal{D}|^{1.14} \times 2^{-0.14j}) = O(|\mathcal{D}|^{1.14}),$$

as the maximum level of the tree is $\log |\mathcal{D}|/S_L$ and the geometric series with $2^{-0.14} < 1$ converges to a constant.

4.5.3 Query Time Complexity of MBI. For TkNN, we analyze the query time complexity of MBI by dividing it into two cases: when $\tau \leq 0.5$ and when $\tau > 0.5$. The following lemma is for analyzing the query time complexity when $\tau \leq 0.5$, indicating that the maximum number of blocks processed in this case is two.

LEMMA 1. *In MBI, if $\tau \leq 0.5$, the maximum number of blocks processing a query is 2.*

PROOF. Consider a query with a time window of (t_s, t_e) . Let $\mathcal{B}_p$ be the first block such that its left child block $\mathcal{B}_l$ and right child block $\mathcal{B}_r$ partition the query time window into two. If the ratio of the query time window to the total time window of $\mathcal{B}_p$ exceeds τ , the query is processed in the single block $\mathcal{B}_p$, thereby satisfying the lemma trivially. If the ratio is less than τ , the query time window is divided into two, (t_s, t_m) and (t_m, t_e) , by $\mathcal{B}_l$ and $\mathcal{B}_r$ respectively. We first show that, for the time window (t_m, t_e) , the query is processed either in $\mathcal{B}_r$ or in exactly one of its descendants. We define α as the ratio of the query time window (t_m, t_e) to the total time window of $\mathcal{B}_r$. If $\alpha \geq \tau$, the query is processed in $\mathcal{B}_r$. If $\alpha < \tau$, the query time window (t_m, t_e) does not overlap with the right child of $\mathcal{B}_r$. This is because the right child of $\mathcal{B}_r$'s time window starts from the midpoint of $\mathcal{B}_r$, and $\tau < 0.5$. In such cases, we repeat this process in the left child of $\mathcal{B}_r$. Accordingly, for the time window (t_m, t_e) , the query is processed either in $\mathcal{B}_r$ if $\alpha \geq \tau$ or in the i th left child block of $\mathcal{B}_r$ if $\tau/2^i \leq \alpha < \tau/2^{i-1}$ for any integer $i \geq 0$. As the query time window (t_s, t_m) is symmetric to (t_m, t_e) , it is processed in $\mathcal{B}_l$ or in precisely one of its descendants in a similar manner, and the lemma holds. $\square$

Now, we prove the query time complexity in MBI when $\tau \leq 0.5$ in the following theorem.

THEOREM 1. *The query time complexity in MBI is $O(\log |\mathcal{D}[t_s : t_e]|/\tau + k/\tau)$ when $\tau \leq 0.5$ and the query time window is (t_s, t_e) where $|\mathcal{D}[t_s : t_e]|$ is the number of vectors in $\mathcal{D}[t_s : t_e]$.*

PROOF. For n vectors, the time complexity of graph-based 1NN search is $O(\log n)$ [16], and for k NN search, the time complexity is $O(\log n + k)$ considering that k searches are performed. Let m be the number of vectors within the query time window. We assume that both m and n are significantly larger than k . Then, the time complexity of SF for TkNN query is $O(\log n + kn/m)$ as the expected number of vectors to be checked is kn/m .

The query time window covers at least τ fraction of every block processing the query in MBI. The number of vectors in each block is less than or equal to $|\mathcal{D}[t_s : t_e]|/\tau$. Consequently, the time complexity to process a query using SF in a single block is $O(\log |\mathcal{D}[t_s : t_e]|/\tau + k/\tau)$. According to Lemma 1, if $\tau \leq 0.5$, the number of blocks processing the query is at most 2. Therefore, the query time complexity of MBI is $O(\log |\mathcal{D}[t_s : t_e]|/\tau + k/\tau)$. $\square$

The following lemma is for the analysis of the query time complexity of MBI when $\tau > 0.5$. For this lemma, we first define an **Internal Left-Aligned Query (ILAQ) block** of a query as a block satisfying the following conditions: (1) it is not a leaf block, (2) this block covers the query time window, and (3) the earliest timestamp of the block is identical to the start timestamp of the query.

LEMMA 2. *MBI with an ILAQ block at the root processes the query using only one block per level, except at the leaf level where up to two blocks are used.*

PROOF. We prove the lemma by induction. Consider a query with time window (t_s, t_e) and an ILAQ block $\mathcal{B}_p = (\mathcal{D}_p, \mathcal{G}_p)$ of the query. As the base case, assume the level of $\mathcal{B}_p$ is 1, i.e., the children of $\mathcal{B}_p$ are leaf blocks. If $|\mathcal{D}[t_s : t_e]|/|\mathcal{D}_p| > \tau$, MBI processes the query on $\mathcal{B}_p$, and the child blocks are not used. In the other case, the two children are used for the query instead of $\mathcal{B}_p$. Thus, the lemma is true in the base case. We now prove that the lemma is true when the level of $\mathcal{B}_p$ is $i + 1$ with the following inductive hypothesis: the lemma holds for every ILAQ block having level i or less. If $|\mathcal{D}[t_s : t_e]|/|\mathcal{D}_p| > \tau$, the lemma is trivially true because the query is wholly processed on $\mathcal{B}_p$. Let $\mathcal{B}_l$ and $\mathcal{B}_r$ be the left and right child blocks of $\mathcal{B}_p$. If $0.5 < |\mathcal{D}[t_s : t_e]|/|\mathcal{D}_p| \leq \tau$, the query is divided into two whose time windows are (t_s, t_m) and (t_m, t_e) , respectively, where t_m is the earliest timestamp of $\mathcal{B}_r$. The query of (t_s, t_m) is processed by $\mathcal{B}_l$ as $\mathcal{D}_l = \mathcal{D}[t_s : t_m]$, and no descendants of $\mathcal{B}_l$ are used. $\mathcal{B}_r$ satisfies the conditions to be an ILAQ block for the query of (t_m, t_e) , thus, the subtree having $\mathcal{B}_r$ as the root follows the lemma. Meanwhile, $\mathcal{B}_p$ itself is not used, as it is not a leaf block and $|\mathcal{D}[t_m : t_e]|/|\mathcal{D}_r| < |\mathcal{D}[t_s : t_e]|/|\mathcal{D}_p| \leq \tau$, where $|\mathcal{D}_r|$ is the number of vectors in $\mathcal{B}_r$. Thus, the lemma follows as only one block is used at level i . If $|\mathcal{D}[t_s : t_e]|/|\mathcal{D}_p| \leq 0.5$, the lemma also trivially holds because $\mathcal{B}_r$ is not used as it contains no vectors belonging to the query time window and $\mathcal{B}_l$ is an ILAQ block of the query. Finally, by the base case and the induction step, the lemma is proven to be true. $\square$

Using this lemma, we analyze the query time complexity of MBI when $\tau > 0.5$ in the following theorem.

THEOREM 2. *When $\tau > 0.5$ and the query time window is (t_s, t_e) , the query time complexity in MBI is $O(\log^2 |\mathcal{D}[t_s : t_e]|/\tau + (k/\tau) \cdot \log |\mathcal{D}[t_s : t_e]|/\tau)$ where $|\mathcal{D}[t_s : t_e]|$ is the number of vectors in $\mathcal{D}[t_s : t_e]$.*

PROOF. Let $\mathcal{B}_p$ be the first block such that its left child block $\mathcal{B}_l = (\mathcal{D}_l, \mathcal{G}_l)$ and right child block $\mathcal{B}_r = (\mathcal{D}_r, \mathcal{G}_r)$ divide the query time window into (t_s, t_m) , and (t_m, t_e) . Then, $\mathcal{B}_r$ is an ILAQ block for the query of time window (t_m, t_e) . Among $\mathcal{B}_r$ and its descendants, the largest block used for the query of (t_m, t_e) is not larger than $|\mathcal{D}[t_s, t_e]| \cdot \tau$. By Lemma 2, the worst case is that the descendants of the largest block are used for every level. As k NN search requires $O(\log n + k/\tau)$ for n vectors

and threshold τ , we calculate the complexity for the query of (t_m, t_e) as follows:

$$\sum_{i=1}^{\log \frac{|\mathcal{D}[t_s:t_e]|}{S_L \tau}} \log \frac{|\mathcal{D}[t_s:t_e]|}{2^i \tau} + \frac{k}{\tau},$$

and this is bounded by $O(\log^2 |\mathcal{D}[t_s:t_e]|/\tau + (k/\tau) \cdot \log |\mathcal{D}[t_s:t_e]|/\tau)$. Finally, since the queries for the time windows (t_s, t_m) and (t_m, t_e) have a symmetrical structure, the overall query time complexity remains the same, i.e., the complexity when $\tau > 0.5$ is $O(\log^2 |\mathcal{D}[t_s:t_e]|/\tau + (k/\tau) \cdot \log |\mathcal{D}[t_s:t_e]|/\tau)$. $\square$

When $\tau \leq 0.5$, MBI theoretically shows a better query time complexity compared to when $\tau > 0.5$, which is well demonstrated in Figure 11. Meanwhile, since the above analyses are about worst cases, in some cases, the performance can be better when $\tau > 0.5$ in practice.

In m -AkNN, the pattern of blocks used for queries is much more complex, making it very difficult to analyze query time for each block separately. Thus, we analyze only a rough upper bound in this case. Given a m -AkNN query $q = (w, k, \dot{a}, \ddot{a})$, $|\mathcal{D}[\dot{a}:\ddot{a}]|$ is the number of vectors in the query range. Then, the upper bound of the query time complexity is as follows:

$$O\left(\frac{|\mathcal{D}[\dot{a}:\ddot{a}]|}{S_L \tau} \left(\log S_L + \frac{k}{\tau}\right)\right),$$

which means the case where only leaf blocks are used. Note that, as observed in the experiments in Section 5.5.1, it operates significantly more efficiently on real-world data.

5 Experiments

In this section, we experimentally evaluate MBI to show its effectiveness. We focus on answering the following questions:

- Q1. *Search performance.* Does MBI provide the best search performance (Section 5.2) for TkNN search?
- Q2. *Scalability.* How well does MBI scale up in terms of the data size for TkNN search (Section 5.3)?
- Q3. *Data insertion time.* How efficiently can new data be inserted into MBI (Section 5.4)?
- Q4. *Effect of parameters.* How does MBI's parameters S_L and τ affect the performance of MBI for TkNN search (Section 5.4)?
- Q5. *Extension to m -AkNN.* How effectively can MBI be extended to support the m -AkNN problem (Section 5.5)?

5.1 Experimental Settings

5.1.1 Machine. All experiments in this article are performed on a machine equipped with AMD Ryzen 73800XT 8-Core Processor and 128 GB of DDR4 RAM. Ubuntu 20.04.1 LTS is installed as the operating system. All codes are written in Rust, and compiled using Cargo v1.66.1 with option “--release.”

5.1.2 Datasets. We use six datasets for the TkNN search experiments and two datasets for the m -AkNN search experiments, summarized in Table 2. MovieLens is a set of movies, each represented as a 32-dimensional vector with its release year as the timestamp. The vectors are generated from users' ratings by a matrix factorization method. The **Communication, Ocean and Meteorological Satellite (COMS)** consists of weather satellite images captured by the COMS, which is the first geostationary multi-purpose satellite of South Korea. We represent each image as a 128-dimensional vector using a convolutional autoencoder. The date and time each image is

Table 2. The Summary of Datasets

Problems	Datasets	# Items		Dim.	Distance	Source
		Train	Test			
TkNN	MovieLens	57,571	200	32	Angular	GroupLens [19]
	COMS	291,180	200	128	Angular	KMA [31]
	GloVe-100	1,183,514	10,000	100	Angular	Pennington et al. [45]
	SIFT1M	1,000,000	10,000	128	Euclidean	Jégou et al. [27]
	GIST1M	1,000,000	1,000	960	Euclidean	
	DEEP1B	9,990,000	10,000	96	Angular	Babenko and Lempitsky [5]
m -AkNN	COMS-GTS	25,544	200	128	Angular	KMA
	CORD-19	49,800	200	728	Angular	Wang et al. [56]

GTS, Global Telecommunication System.

captured is used as the timestamp. We also use four public large datasets that originally do not have timestamps: GloVe-100, SIFT1M, GIST1M, and DEEP1B. We consider the index of each item as its virtual timestamp. GloVe-100 is a large word embedding dataset where each word is represented as a 100-dimensional vector. SIFT1M, GIST1M, and DEEP1B are large image descriptor datasets, where images are represented as 128-dimensional, 960-dimensional, and 96-dimensional vectors, respectively.

COMS-Global Telecommunication System (GTS) is the COMS dataset combined with the average temperature and humidity in South Korea. These values, along with timestamps, serve as filtering properties when $m = 2, 3$. The temperature and humidity values are derived from the GTS, collected between October 2010 and March 2020. The dataset size is reduced since only data points with the additional properties (i.e., temperature and humidity) are used. CORD-19 is a dataset consisting of academic papers on COVID-19, SARS-CoV-2, and related coronaviruses, where each paper is represented as a 768-dimensional vector using the SPECTER [11] model. Using the PubMed IDs in the dataset and the PubMed API [41], we acquire the citation counts for each paper from 10 October to 17 October 2024. We use these citation counts and the publication date of each paper as filtering attributes. Among papers with at least one citation, 50,000 papers are sampled.

5.1.3 Parameter Settings. For each block of MBI and SF, k NN graph-based index structure, which is constructed by NNDescent, and the graph search algorithm in Algorithm 3 are used for TkNN and m -AkNN queries. As the number of neighbors in the graph and the number M_C of max candidates in the search algorithm significantly affect the search performance, we find the optimal values for each dataset by grid search and use them for our experiments. The search algorithm employed by both SF and MBI has another parameter ϵ , which is involved in how far the candidate is extended during the search. The parameter ϵ balances the tradeoff between query speed and recall. We vary the value of ϵ in increments of 0.02, ranging from 1 to 1.4, and present the optimal based on the Pareto frontier. We set the parameter k , representing the number of nearest neighbors to find, to different values (10, 50, and 100) to see how k affects the performance. If not specified, the default value for k is 10. MBI has two own parameters: the threshold τ used for selecting blocks and the leaf size S_L . We use different τ values for each dataset; the values are selected to have the best query speed from a range of 0.1–0.9 (see Section 5.4.2). The default values for S_L are set according to the scale of each dataset considering the index size. The effect of S_L on the performance of MBI is studied in Section 5.4.1 in detail. The default parameters are summarized in Table 3.

Table 3. Default Parameters

Datasets	Graph Search Parameters				MBI Parameters	
	# Neighbors	M_C	ϵ	k	τ	S_L
MovieLens	96	192			0.5	3,550
COMS	256	256			0.2, 0.4	1,000
GloVe-100	256	256			0.2, 0.7	36,000
SIFT1M	128	128	$1 - 1.4$	10 (default),	0.3, 0.5	15,625
GIST1M	512	512	(by 0.02)	50, 100	0.3, 0.5	15,625
DEEP1B	64	64			0.2, 0.5	78,000
COMS-GTS	256	256			0.2, 0.4	1,000
CORD-19	128	128			0.5	1,000

5.2 TkNN Search Performance

Figure 7 shows the TkNN search performance of MBI, BSBF, and SF according to the size of the query time window when $\text{recall}@k$ is 0.995 with k set to values of 10, 50, and 100. Here, $\text{recall}@k$ measures the quality of an approximate answer $\hat{A}$ compared to the true answer A , and is defined as follows:

$$\text{recall}@k(\hat{A}, A) = \frac{|\hat{A} \cap A|}{k}.$$

We sample 200 vectors randomly for each dataset to form queries. These queries are excluded during the indexing process. The start and end times (i.e., t_s and t_e) of the query time window are randomly determined to cover a fraction of the entire data. This process is repeated five times for fractions ranging from 1% to 95%. The x -axis represents the fraction, i.e., $\mathcal{D}[t_s : t_e] / |\mathcal{D}|$, and the y -axis represents the number of queries per second. For MBI, we choose the values of τ that result in the highest queries per second (see Section 5.4). MBI demonstrates superior query speed compared to BSBF and SF in all datasets regardless of the length of the query time window. MBI even outperforms a hypothetical method that selects the faster of BSBF and SF in most cases, showing up to 10.88 times faster speed. The tendency is similar for all datasets and the three values of k , while the query speed decreases as k increases. This figure also clearly shows that BSBF gets slower when the query time window gets longer, and SF gets slower when the query time window gets shorter, as mentioned in Sections 3.2.1 and 3.2.2.

Figure 8 shows the $\text{recall}@10$ and queries per second of MBI, BSBF, and SF with various ϵ values from 1 to 1.4 when $|\mathcal{D}[t_s : t_e]| / |\mathcal{D}|$ is 10%, 30%, and 80%. COMS is used. Just like when $\text{recall}@10$ is 0.995, we observe that similar patterns mentioned above exist across other $\text{recall}@10$ values as well.

5.3 Scalability in TkNN Search

Figure 9 shows the data scalability of MBI. SIFT1M dataset is used. The slope in the plot indicates the scalability as both axes are on a log scale. Figure 9(a) shows the indexing time of MBI and SF according to the data size. The slope of MBI gradually decreases as the data size increases, showing a value of 1.29. This implies that to index double the data in MBI, it requires $2^{1.29} = 2.45$ times the indexing time. MBI exhibits a logarithmic increase in indexing time complexity when the dataset is small due to its hierarchical structure. However, as the dataset grows, the slope decreases and is expected to approach 1.14. We note that MBI trades off a slight increase in indexing time to significantly improve the speed of queries, which are more frequently executed. Besides, since each block in MBI is independently created, it is easily parallelized. By generating blocks in parallel,

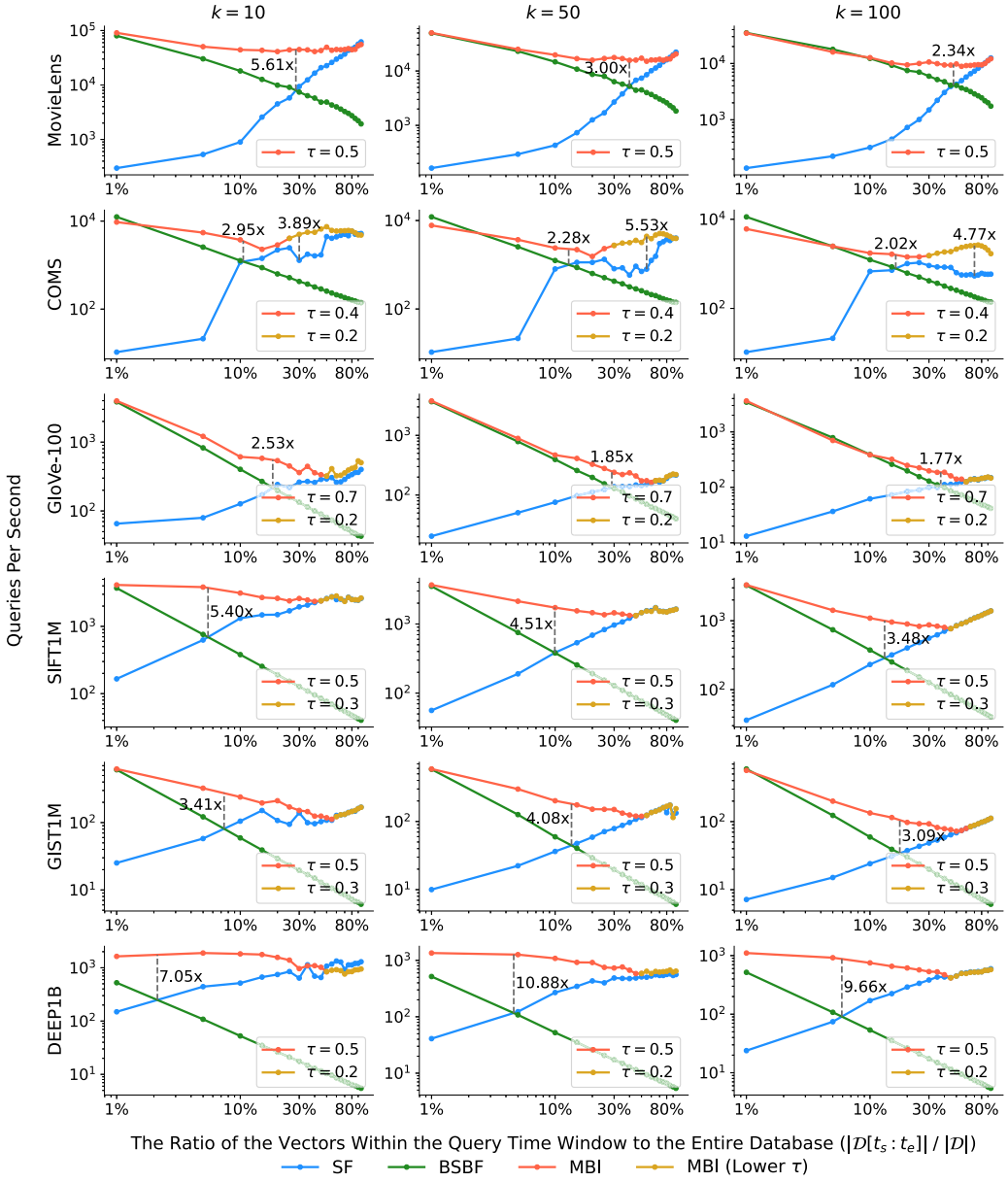


Fig. 7. The ratio of the query time window vs. the number of queries per second when recall@ k is set to 0.995 and k is set to 10, 50, and 100.

the indexing time of MBI is comparable to that of SF, resulting in a decrease of the indexing time by up to 5.08 times. Figure 9(b) shows the index sizes of MBI and SF according to the data size. Similarly to the indexing time results, the slope of MBI gradually decreases to 1.29 as the size of the data increases. This result matches the theoretical analysis in Section 4.5.1. The index sizes of other datasets are listed in Table 4. The number in a parenthesis indicates the relative index size compared to the input data size.

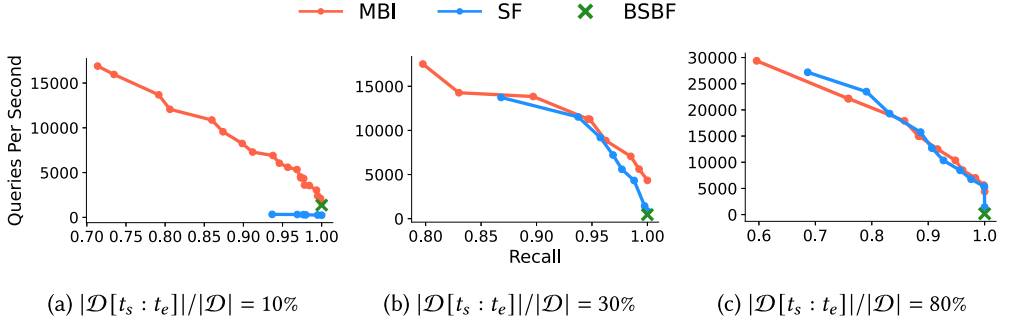


Fig. 8. The recall@10 vs. queries per second on COMS dataset. The ratios of the query time window are set to 10%, 30%, and 80%.

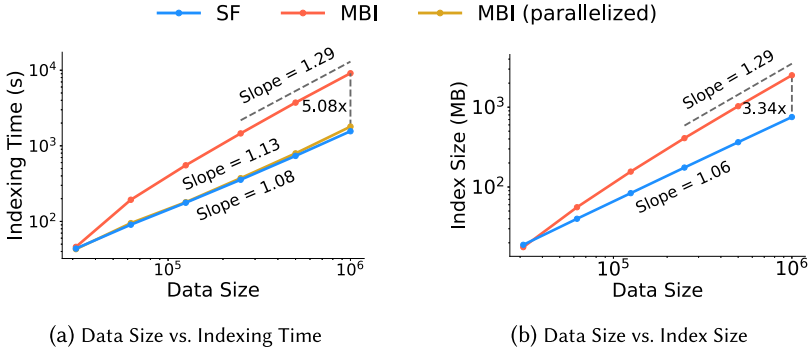


Fig. 9. The scalability of MBI and SF on SIFT1M dataset.

Table 4. Index Sizes of MBI and SF

Datasets	Input Data Size (GB)	Index Sizes (GB)	
		MBI	SF
MovieLens	0.01	0.06 (6.08 $\times$)	0.02 (1.90 $\times$)
COMS	0.15	0.91 (6.35 $\times$)	0.27 (1.74 $\times$)
GloVe-100	0.48	3.95 (8.72 $\times$)	1.21 (2.49 $\times$)
SIFT1M	0.53	2.09 (4.28 $\times$)	0.80 (1.53 $\times$)
GIST1M	3.85	7.73 (2.15 $\times$)	4.64 (1.21 $\times$)
DEEP1B	3.92	18.24 (5.00 $\times$)	6.10 (1.56 $\times$)

5.4 Effect of Parameters

MBI has two own parameters: an indexing parameter S_L of the leaf block size and a query parameter τ for choosing the search block. We examine the effect of each parameter on the performance of MBI.

5.4.1 Effect of Leaf Size S_L . As the leaf size S_L affects the index size, the indexing time, and the query speed, we compare them in Figure 10. MovieLens is used. Figure 10(a) shows the cumulative indexing time measured when data are incrementally inserted into MBI. A lower S_L tends to require a higher indexing time, although the difference is not significant. As analyzed in Section 4.5.2, the results approximate $n^{1.14}$ with respect to the number of inserted data n . The results demonstrate a

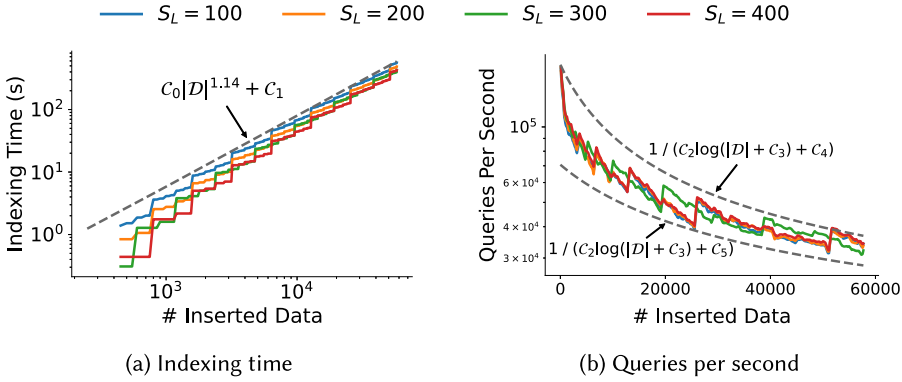


Fig. 10. The indexing time and queries per second with different leaf sizes on MovieLens dataset. C_0 to C_5 are set as follows: $2.2 \cdot 10^{-3}$, 0.05, $2.61 \cdot 10^{-5}$, 45,000, $-2.74 \cdot 10^{-4}$, and $-2.655 \cdot 10^{-4}$.

consistent difference depending on the size of S_L , while maintaining a similar pattern as the data size increases. Figure 10(b) shows the query speed measured each time a vector is appended to MBI. The queries are conducted with the size of the time window randomly set from 5% to 95% of the current data size. The query speed tends to decrease slightly as S_L increases, but the difference is almost negligible. The query speed exhibits a kind of zigzag patterns but shows a steady decrease in the overall trend, which matches well with the results analyzed in the Section 4.5.3. The sudden increase in query speed occurs when MBI creates a complete tree. Overall, smaller S_L improves query performance for narrow time windows at the cost of slightly higher indexing time and index size, whereas larger S_L reduces indexing overhead with minimal query slowdown. In practice, setting S_L such that the resulting tree depth is between 5 and 10 provides a good balance among build time, index size, and query speed.

5.4.2 Effect of Threshold τ . Figure 11 shows the query speed of MBI with various τ values from 0.1 to 0.9 according to the size of the query time window when recall@10 is 0.995. BSBF and SF are also displayed for reference. When $\tau > 0.5$, the query speed tends to decrease as τ increases. This is because the search is performed in many blocks if τ values are high. When $\tau \leq 0.5$, MBI performs well with a high τ when the query time window is low, and conversely, performs well with a low τ when the query time window is high. This is because it is guaranteed that queries are processed in two or fewer blocks when τ is 0.5 or less, as we prove it in Lemma 1, and the lower the τ value, the more likely it is to be processed in larger blocks. The results show that the performance is satisfactory when τ is around 0.5 across all datasets. Therefore, we recommend these values when no prior information is available. If possible, one can compute the optimal τ for each query interval experimentally beforehand, and use the pre-computed τ at runtime.

5.5 Extension to m -AkNN

This section evaluates the performance of MBI on the m -AkNN problem for $m = 2, 3$ based on experimental results.

5.5.1 m -AkNN Search Performance. We randomly sample 200 vectors from each dataset to generate queries, following the same experimental setup as MBI for TkNN. These queries are excluded during the indexing process. Unlike the queryset used in TkNN, we define the filtering range based on filtering attributes by randomly setting the search range. Instead of using the query

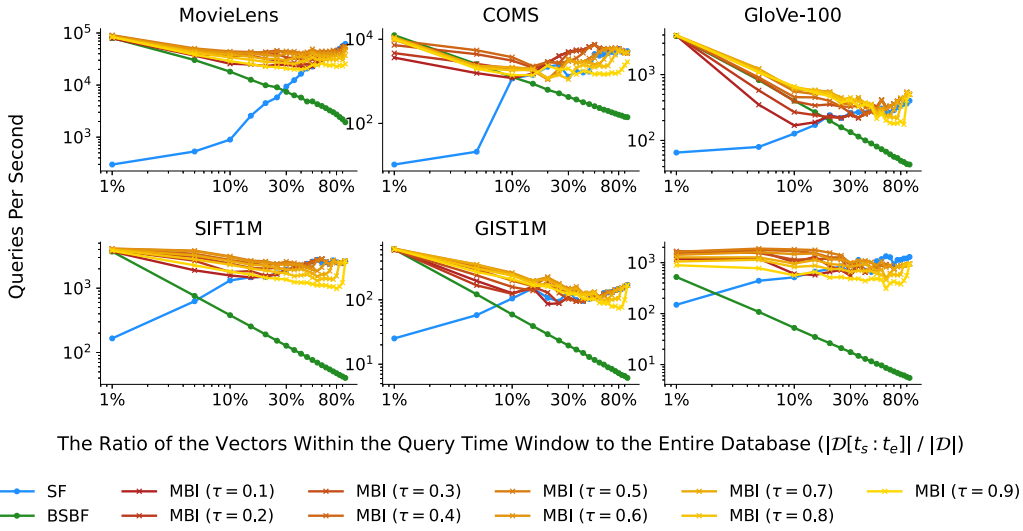


Fig. 11. The ratio of the query time window vs. the number of queries per second when recall@10 is set to 0.995. MBI with different τ from 0.1 to 0.9 is used.

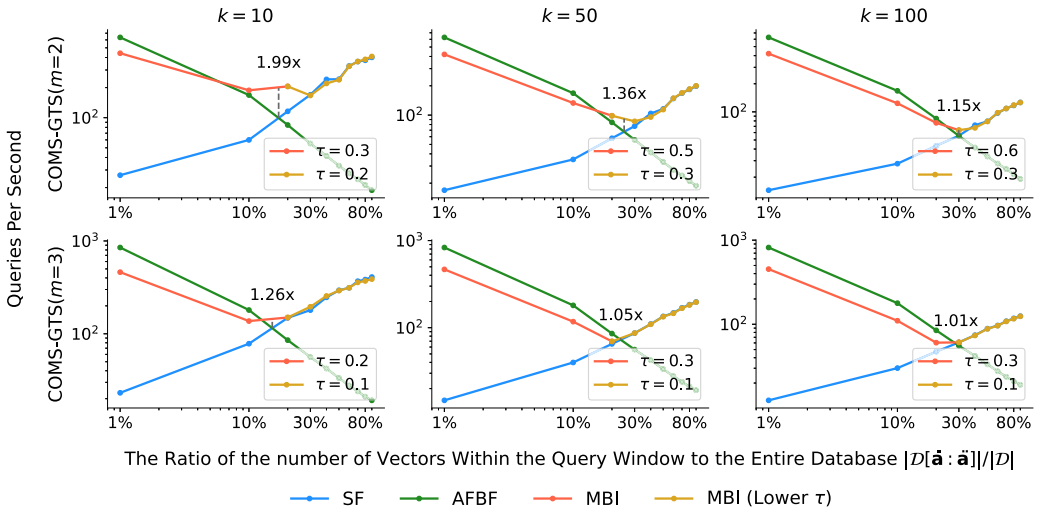


Fig. 12. The ratio of the number of vectors within the query window to the entire database vs. the number of queries per second on COMS-GTS dataset when recall@ k is set to 0.95.

time window ratio, we ensure that the ratio of the number of vectors within the search range to the entire database remains fixed.

Figure 12 shows the m - Ak NN search performance of MBI, AFBF, and SF according to the number of vectors within the query window when recall@ k is 0.95 with k set to values of 10, 50, and 100. The COMS-GTS dataset is used. For recall@ k is set to 0.95, which is lower than 0.995 that used for TkNN. This adjustment is made to allow a comparison with SF, which exhibits relatively lower recall for $m = 2, 3$ (see Figure 13). Each experiment is repeated three times under the given parameter settings. The x -axis represents the ratio of the number of vectors within the query window to the

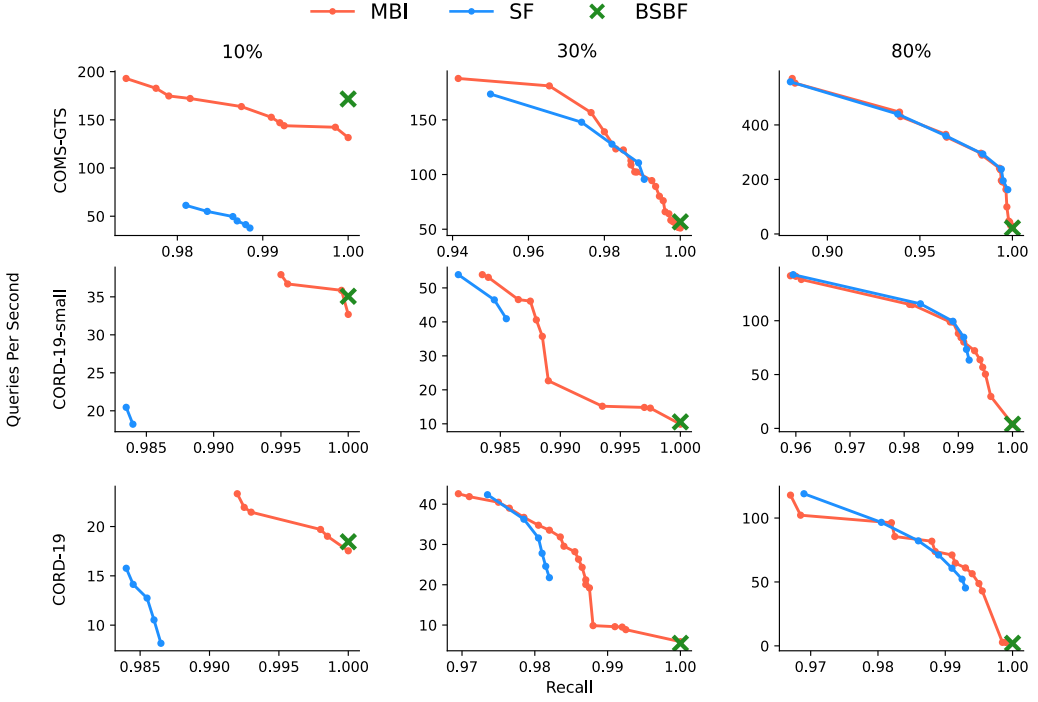


Fig. 13. The recall@10 vs. queries per second on COMS-GTS and CORD-19 datasets when $m = 2$. The ratios of the number of vectors satisfying attribute constraints are set to 10%, 30%, and 80%. CORD-19-small is a subset of CORD-19, sampled to match the same number of data points to COMS-GTS (25,544).

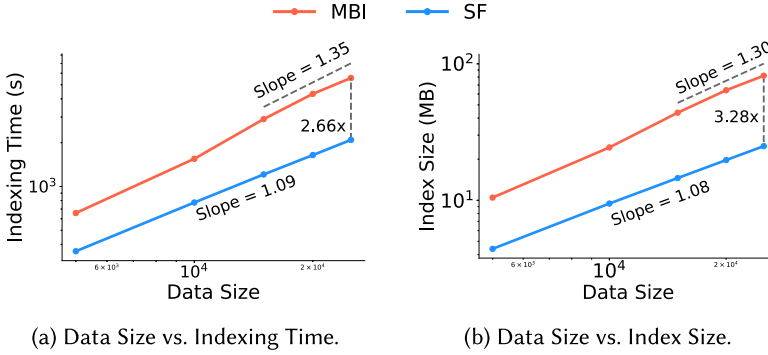


Fig. 14. The scalability of MBI and SF on COMS-GTS dataset when $m = 2$.

entire database, $|\mathcal{D}[a : a]|/|\mathcal{D}|$. The y-axis represents the number of queries per second. As in the case of TkNN search, MBI maintains stable query processing speed across all query filtering ranges in the m -AkNN problem, while AFBF and SF exhibit significant performance degradation depending on the filtering ratio.

Figure 13 shows the recall@10 and queries per second of MBI, AFBF, and SF with various ϵ values from 1 to 1.4 when $|\mathcal{D}[a : a]|$ is 10%, 30%, and 80% like Figure 8. MBI achieves higher query processing speed than AFBF and higher recall@ k than SF. This is because MBI searches fewer vectors compared to AFBF and filters out fewer candidates compared to SF. In the case of the

Table 5. Index Size of MBI for m -AkNN and SF

Datasets	Input Data Size (GB)	m	Index Size (GB)	
			MBI	SF
COMS-GTS	0.01	2	0.08 (5.66 $\times$)	0.03 (2.29 $\times$)
		3	0.07 (4.59 $\times$)	0.03 (2.30 $\times$)
CORD-19-small	1.57	2	2.11 (1.34 $\times$)	1.65 (1.05 $\times$)
CORD-19-large	3.05	2	4.23 (1.38 $\times$)	3.21 (1.05 $\times$)

CORD-19 dataset, which has a higher dimensionality than the COMS-GTS dataset, MBI achieves significantly higher accuracy and query processing speed than SF when $|\mathcal{D}[a : a]| \leq 10\%$.

5.5.2 Scalability in m -AkNN Search. Figure 14 illustrates the indexing time and index size for m -AkNN as the data size increases when $m = 2$. The COMS-GTS dataset is used. The results exhibit a similar trend to the scalability experiment on TkNN in Section 5.2. For reference, Table 5 presents the index size for the COMS-GTS and CORD-19 datasets.

6 Conclusion

In this article, we propose MBI, a novel and efficient indexing method for TkNN queries on high-dimensional and time-accumulating dataset. We devise an incremental hierarchical index structure that organizes data into blocks based on their timestamps, ensuring MBI's query processing remains efficient regardless of the query time window's length. Additionally, to apply MBI to multiple numerical attributes, we newly define the m -AkNN problem, and extend MBI by adopting spatial partitioning structures such as quadrees and octrees to solve it. In our theoretical analysis of MBI's efficiency, we find that the index size is $O(|\mathcal{D}| \log |\mathcal{D}|)$, and it requires $O(|\mathcal{D}|^{1.14})$ time to index the data, suggesting an amortized data insertion time of $O(|\mathcal{D}|^{0.14})$ where $|\mathcal{D}|$ is the number of timestamped vectors in database $\mathcal{D}$. Identically, for m -AkNN, the index size is $O(|\mathcal{D}| \log |\mathcal{D}|)$ and the total time to index the data are $O(|\mathcal{D}|^{1.14})$. The query time is $O(\log |\mathcal{D}| + k/\tau)$ for TkNN queries, and $O(|\mathcal{D}|/S_L \tau (\log S_L + k/\tau))$ for m -AkNN queries, where k is the number of query results, and τ is a threshold parameter of MBI used for block selection. Experimental results show that in the case of TkNN, MBI is up to 10.88 times faster than a hypothetical method that selects the faster of conventional methods: BSBF and SF. For m -AkNN, MBI is up to 1.99 times faster and exhibits higher accuracy compared to a hypothetical baseline.

Limitations and Future Work

While the proposed MBI framework demonstrates promising efficiency for both TkNN and m -AkNN problems, certain design constraints remain due to the scope of the current work. For the TkNN case, the indexing strategy assumes that data arrive in non-decreasing timestamp order, and thus does not accommodate deletions or insertions with arbitrary timestamps. Similarly, in the m -AkNN setting, the current design focuses on static datasets, where dynamic updates, such as arbitrary insertions or deletions, are not considered. One possible extension is to support sequential removal of outdated data. For the TkNN problem, this could be achieved by progressively removing the leftmost blocks while maintaining the tree structure of MBI, thereby enabling sliding-window style deletions without significant restructuring cost. In addition, although empirical results validate the efficiency of the approach, a formal optimality analysis for both TkNN and m -AkNN remains an open challenge. Addressing these aspects could further enhance the applicability and theoretical robustness of MBI, and thus represent important directions for future research.

References

- [1] Fabien André, Anne-Marie Kermarrec, and Nicolas Le Scouarnec. 2017. Accelerated nearest neighbor search with quick ADC. In *Proceedings of the 2017 ACM on International Conference on Multimedia Retrieval (ICMR '17)*. ACM, 159–166. DOI: <https://doi.org/10.1145/3078971.3078992>
- [2] Fabien André, Anne-Marie Kermarrec, and Nicolas Le Scouarnec. 2021. Quicker ADC: Unlocking the hidden potential of product quantization with SIMD. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 43, 5 (2021), 1666–1677. DOI: <https://doi.org/10.1109/TPAMI.2019.2952606>
- [3] Martin Aumüller, Tobias Christiani, Rasmus Pagh, and Michael Vesterli. 2019. PUFFINN: Parameterless and universally fast finding of nearest neighbors. In *Proceedings of the 27th Annual European Symposium on Algorithms (ESA '19)*, Vol. 144. Schloss Dagstuhl—Leibniz-Zentrum für Informatik, Article 10, 1–16.
- [4] Franz Aurenhammer, Rolf Klein, and Der-Tsai Lee. 2013. *Voronoi Diagrams and Delaunay Triangulations*. World Scientific Publishing Company.
- [5] Artem Babenko and Victor Lempitsky. 2016. Efficient indexing of billion-scale datasets of deep descriptors. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR '16)*, 2055–2063.
- [6] Norbert Beckmann, Hans-Peter Kriegel, Ralf Schneider, and Bernhard Seeger. 1990. The R*-tree: An efficient and robust access method for points and rectangles. In *Proceedings of the 1990 ACM SIGMOD International Conference on Management of Data (SIGMOD '90)*. ACM, 322–331. DOI: <https://doi.org/10.1145/93597.98741>
- [7] Erik Bernhardsson. 2025. Benchmarking Nearest Neighbors. Retrieved from <https://github.com/erikbern/ann-benchmarks>
- [8] Alina Beygelzimer, Sham M. Kakade, and John Langford. 2006. Cover trees for nearest neighbor. In *Proceedings of the 23rd International Conference on Machine Learning (ICML '06)*, Vol. 148. ACM, 97–104. DOI: <https://doi.org/10.1145/1143844.1143857>
- [9] Qi Chen, Bing Zhao, Haidong Wang, Mingqin Li, Chuanjie Liu, Zengzhong Li, Mao Yang, and Jingdong Wang. 2021. SPANN: Highly-efficient billion-scale approximate nearest neighbor search. arXiv:2111.08566. Retrieved from <https://arxiv.org/abs/2111.08566>
- [10] Minyoung Choe, Geon Lee, Changhun Han, Suji Kim, Woong Hu, Hyebeen Hwang, Geunseok Park, Byeongyeon Kim, Hyesook Lee, Ha-Myung Park, et al. 2025. SkySearch: Satellite video search at scale. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD '25)*. ACM, New York, NY, 4353–4364. DOI: <https://doi.org/10.1145/3711896.3737263>
- [11] Arman Cohan, Sergey Feldman, Iz Beltagy, Doug Downey, and Daniel Weld. 2020. SPECTER: Document-level representation learning using citation-informed transformers. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*. Dan Jurafsky, Joyce Chai, Natalie Schluter, and Joel Tetreault (Eds.), Association for Computational Linguistics, 2270–2282. DOI: <https://doi.org/10.18653/v1/2020.acl-main.207>
- [12] Sanjoy Dasgupta and Yoav Freund. 2008. Random projection trees and low dimensional manifolds. In *Proceedings of the 40th Annual ACM Symposium on Theory of Computing (STOC '08)*. ACM, 537–546. DOI: <https://doi.org/10.1145/1374376.1374452>
- [13] Wei Dong, Moses Charikar, and Kai Li. 2011. Efficient k-nearest neighbor graph construction for generic similarity measures. In *Proceedings of the 20th International World Wide Web Conference (WWW '11)*. ACM, 577–586. DOI: <https://doi.org/10.1145/1963405.1963487>
- [14] Jerome H. Friedman, Jon Louis Bentley, and Raphael A. Finkel. 1977. An algorithm for finding best matches in logarithmic expected time. *ACM Transactions on Mathematical Software* 3, 3 (1977), 209–226. DOI: <https://doi.org/10.1145/355744.355745>
- [15] Cong Fu and Deng Cai. 2016. EFANNA: An extremely fast approximate nearest neighbor search algorithm based on kNN graph. arXiv:1609.07228. Retrieved from <http://arxiv.org/abs/1609.07228>
- [16] Cong Fu, Chao Xiang, Changxu Wang, and Deng Cai. 2019. Fast approximate nearest neighbor search with the navigating spreading-out graph. *Proceedings of the VLDB Endowment* 12, 5 (2019), 461–474. DOI: <https://doi.org/10.14778/3303753.3303754>
- [17] Jianyang Gao and Cheng Long. 2023. High-dimensional approximate nearest neighbor search: With reliable and efficient distance comparison operations. *Proceedings of the ACM on Management of Data* 1, 2 (2023), 1–27.
- [18] Aristides Gionis, Piotr Indyk, and Rajeev Motwani. 1999. Similarity search in high dimensions via hashing. In *Proceedings of the 25th International Conference on Very Large Data Bases (VLDB '99)*. Morgan Kaufmann, 518–529.
- [19] GroupLens. 2025. MovieLens. Retrieved from <https://grouplens.org/datasets/movielens/>
- [20] Ruiqi Guo, Philip Guo, Erik Lindgren, Quan Geng, David Simcha, Felix Chern, and Sanjiv Kumar. 2020. Accelerating large-scale inference with anisotropic vector quantization. In *Proceedings of the 37th International Conference on Machine Learning (ICML '20)*, Vol. 119. PMLR, 3887–3896.

- [21] Marios Hadjieleftheriou, George Kollios, Vassilis J. Tsotras, and Dimitrios Gunopulos. 2002. Efficient indexing of spatiotemporal objects. In *Proceedings of the 8th International Conference on Extending Database Technology (EDBT '02)*, Vol. 2287. Springer, 251–268. DOI: https://doi.org/10.1007/3-540-45876-X_17
- [22] Changhun Han, Suji Kim, and Ha-Myung Park. 2024. Efficient proximity search in time-accumulating high-dimensional data using multi-level block indexing. In *Proceedings of the 27th International Conference on Extending Database Technology (EDBT '24)*, 547–558.
- [23] Ben Harwood and Tom Drummond. 2016. FANNG: Fast approximate nearest neighbour graphs. In *Proceedings of the 2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR '16)*. IEEE Computer Society, 5713–5722. DOI: <https://doi.org/10.1109/CVPR.2016.616>
- [24] Tim Ingham. 2021. Over 60,000 Tracks Are Now Uploaded to Spotify Every Day. That's Nearly One per Second. Retrieved from <https://www.musicbusinessworldwide.com/over-60000-tracks-are-now-uploaded-to-spotify-daily-thats-nearly-one-per-second/>
- [25] Masajiro Iwasaki and Daisuke Miyazaki. 2018. Optimization of indexing based on k-nearest neighbor graph for proximity search in high-dimensional data. arXiv:1810.07355. Retrieved from <http://arxiv.org/abs/1810.07355>
- [26] Suhas Jayaram Subramanya, Fnu Devvrit, Harsha Vardhan Simhadri, Ravishankar Krishnawamy, and Rohan Kadekodi. 2019. DiskANN: Fast accurate billion-point nearest neighbor search on a single node. In *Proceedings of the 33rd International Conference on Neural Information Processing Systems (NeurIPS '19)*, Vol. 32. Curran Associates, Inc.
- [27] Herve Jegou, Matthijs Douze, and Cordelia Schmid. 2010. Product quantization for nearest neighbor search. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 33, 1 (2010), 117–128.
- [28] Jeff Johnson, Matthijs Douze, and Hervé Jégou. 2021. Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data*, 3 (2021), 535–547. DOI: <https://doi.org/10.1109/TBDATA.2019.2921572>
- [29] Herve Jégou, Matthijs Douze, and Cordelia Schmid. 2011. Product quantization for nearest neighbor search. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 33, 1 (2011), 117–128. DOI: <https://doi.org/10.1109/TPAMI.2010.57>
- [30] Sang-Hong Kim and Ha-Myung Park. 2023. Efficient distributed approximate k-nearest neighbor graph construction by multiway random division forest. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD '23)*. ACM, New York, NY, 1097–1106. DOI: <https://doi.org/10.1145/3580305.3599327>
- [31] KMA. 2025. GK2A Filesets. Retrieved from <https://data.kma.go.kr/data/rmt/rmtList.do?code=20&pgmNo=63>
- [32] George Kollios, Vassilis J. Tsotras, Dimitrios Gunopulos, Alex Delis, and Marios Hadjieleftheriou. 2001. Indexing animated objects using spatiotemporal access methods. *IEEE Transactions on Knowledge and Data Engineering* 13, 5 (2001), 758–777. DOI: <https://doi.org/10.1109/69.956099>
- [33] Quoc V. Le and Tomás Mikolov. 2014. Distributed representations of sentences and documents. In *Proceedings of the 31st International Conference on Machine Learning (ICML '14)*, Vol. 32. JMLR, 1188–1196. Retrieved from <http://proceedings.mlr.press/v32/le14.html>
- [34] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen Tau Yih, Tim Rocktäschel, et al. 2021. Retrieval-augmented generation for knowledge-intensive NLP tasks. arXiv:2005.11401. Retrieved from <https://arxiv.org/abs/2005.11401>
- [35] Yury Malkov, Alexander Ponomarenko, Andrey Logvinov, and Vladimir Krylov. 2014. Approximate nearest neighbor algorithm based on navigable small world graphs. *Information Systems* 45 (2014), 61–68. DOI: <https://doi.org/10.1016/j.is.2013.10.006>
- [36] Yu A. Malkov and Dmitry A. Yashunin. 2018. Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 42, 4 (2018), 824–836.
- [37] Rosalind B. Marimont and Marvin B. Shapiro. 1979. Nearest neighbour searches and the curse of dimensionality. *IMA Journal of Applied Mathematics* 24, 1 (1979), 59–70.
- [38] Puya Memarzia, Maria Patrou, Md Mahbub Alam, Suprio Ray, Virendra C. Bhavsar, and Kenneth B. Kent. 2019. Toward efficient processing of spatio-temporal workloads in a distributed in-memory system. In *Proceedings of the 20th IEEE International Conference on Mobile Data Management (MDM '19)*. IEEE, 118–127. DOI: <https://doi.org/10.1109/MDM.2019.00-66>
- [39] Marius Muja and David G. Lowe. 2009. Fast approximate nearest neighbors with automatic algorithm configuration. In *Proceedings of the 4th International Conference on Computer Vision Theory and Applications (VISAPP '09)*. INSTICC, 331–340.
- [40] Randal C. Nelson and Hanan Samet. 1986. A consistent hierarchical representation for vector data. In *Proceedings of the 13th Annual Conference on Computer Graphics and Interactive Techniques*, 197–206.
- [41] NLM. 2024. Pubmed for Developers. Retrieved from <https://pmc.ncbi.nlm.nih.gov/tools/developers/>
- [42] NMSC. 2025. Information of GK2A. Retrieved from <https://nm-sc.kma.go.kr/enhome/html/base/cmm/selectPage.do?page=satellite.gk2a.intro>

- [43] Stephen M. Omohundro. 1989. *Five Balltree Construction Algorithms*. International Computer Science Institute Berkeley.
- [44] Rodrigo Paredes and Edgar Chávez. 2005. Using the k -nearest neighbor graph for proximity searching in metric spaces. In *Proceedings of the 12th International Symposium on String Processing and Information Retrieval (SPIRE '05)*, Vol. 3772. Springer, 127–138. DOI: https://doi.org/10.1007/11575832_14
- [45] Jeffrey Pennington, Richard Socher, and Christopher D. Manning. 2014. GloVe: Global vectors for word representation. In *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP '14)*, 1532–1543. Retrieved from <http://www.aclweb.org/anthology/D14-1162>
- [46] D. Pfoser, C. S. Jensen, and Y. Theodoridis. 2000. Novel approaches in query processing for moving object trajectories. In *Proceedings of the International Conference on Very Large Data Bases (VLDB '00)*. VLDB.
- [47] Kriengkrai Porkaew, Iosif Lazaridis, and Sharad Mehrotra. 2001. Querying mobile objects in spatio-temporal databases. In *Proceedings of the 7th International Symposium on Advances in Spatial and Temporal Databases (SSTD '01)*. Springer, 59–78.
- [48] Larissa C. Shimomura, Rafael Seidi Oyamada, Marcos R. Vieira, and Daniel S. Kaster. 2021. A survey on graph-based methods for similarity searches in metric spaces. *Information Systems* 95 (2021), 101507. DOI: <https://doi.org/10.1016/j.is.2020.101507>
- [49] Yufei Tao and Dimitris Papadias. 2001. MV3R-Tree: A spatio-temporal access method for timestamp and interval queries. In *Proceedings of the 27th International Conference on Very Large Data Bases (VLDB '01)*. Morgan Kaufmann, 431–440. Retrieved from <http://www.vldb.org/conf/2001/P431.pdf>
- [50] Jamel Tayeb, Özgür Ulusoy, and Ouri Wolfson. 1998. A quadtree-based dynamic attribute indexing method. *The Computer Journal* 41, 3 (1998), 185–200.
- [51] GMI Research Team. 2025. Youtube Statistics 2025. Retrieved from <https://www.globalmediainsight.com/blog/youtube-users-statistics/>
- [52] Godfried T. Toussaint. 1980. The relative neighbourhood graph of a finite planar set. *Pattern Recognition* 12, 4 (1980), 261–268. DOI: [https://doi.org/10.1016/0031-3203\(80\)90066-7](https://doi.org/10.1016/0031-3203(80)90066-7)
- [53] Trieu H. Trinh, Minh-Thang Luong, and Quoc V. Le. 2019. Selfie: Self-supervised pretraining for image embedding. arXiv:1906.02940. Retrieved from <http://arxiv.org/abs/1906.02940>
- [54] Theodoros Tzouramanis, Michael Vassilakopoulos, and Yannis Manolopoulos. 2000. Overlapping linear quadtrees and spatio-temporal query processing. *The Computer Journal* 43, 4 (2000), 325–343. DOI: <https://doi.org/10.1093/comjnl/43.4.325>
- [55] Dongjing Wang, ShuiGuang Deng, Xin Zhang, and Guandong Xu. 2016. Learning music embedding with metadata for context aware recommendation. In *Proceedings of the 2016 ACM International Conference on Multimedia Retrieval (ICMR '16)*. ACM, 249–253. DOI: <https://doi.org/10.1145/2911996.2912045>
- [56] Lucy Lu Wang, Kyle Lo, Yoganand Chandrasekhar, Russell Reas, Jiangjiang Yang, Doug Burdick, Darrin Eide, Kathryn Funk, Yannis Katsis, Rodney Michael Kinney, et al. 2020. CORD-19: The COVID-19 open research dataset. In *Proceedings of the 1st Workshop on NLP for COVID-19 at ACL 2020*. Association for Computational Linguistics. Retrieved from <https://www.aclweb.org/anthology/2020.nlpCOVID19-acl.1>
- [57] Mengzhao Wang, Xiaoliang Xu, Qiang Yue, and Yuxiang Wang. 2021. A comprehensive survey and experimental comparison of graph-based approximate nearest neighbor search. *Proceedings of the VLDB Endowment* 14, 11 (2021), 1964–1978. DOI: <https://doi.org/10.14778/3476249.3476255>
- [58] Patrick Wieschollek, Oliver Wang, Alexander Sorkine-Hornung, and Hendrik P. A. Lensch. 2016. Efficient large-scale approximate nearest neighbor search on the GPU. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR '16)*.
- [59] Jason Wise. 2025. How Many Pictures Are on Instagram in 2025? Retrieved from <https://earthweb.com/how-many-pictures-are-on-instagram/>
- [60] Donghui Yan, Yingjie Wang, Jin Wang, Honggang Wang, and Zhenpeng Li. 2018. K-nearest neighbor search by random projection forests. In *Proceedings of the IEEE International Conference on Big Data (Big Data '18)*. IEEE Computer Society, Los Alamitos, CA, 4775–4781. DOI: <https://doi.org/10.1109/BigData.2018.8622307>
- [61] Peter N. Yianilos. 1993. Data structures and algorithms for nearest neighbor search in general metric spaces. In *Proceedings of the 4th Annual ACM-SIAM Symposium on Discrete Algorithms (SODA '93)*. SIAM, 311–321.
- [62] Weijie Zhao, Shulong Tan, and Ping Li. 2020. SONG: Approximate nearest neighbor search on GPU. In *Proceedings of the 36th IEEE International Conference on Data Engineering (ICDE '20)*. IEEE, 1033–1044. DOI: <https://doi.org/10.1109/ICDE48307.2020.00094>
- [63] Wan-Lei Zhao, Hui Wang, and Chong-Wah Ngo. 2021. Approximate k-NN graph construction: A generic online approach. *IEEE Transactions on Multimedia* 24 (2021), 1909–1921. DOI: <https://doi.org/10.1109/TMM.2021.3073811>

Received 17 March 2025; revised 10 November 2025; accepted 6 January 2026